

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ

—o0o—



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

THIẾT KẾ HỆ THỐNG GIÁM SÁT
NĂNG LƯỢNG ĐIỆN

GVHD: ThS. Bùi Quốc Bảo

Sinh viên thực hiện

MSSV

Nguyễn Xuân Phát

2212529

TP.HỒ CHÍ MINH, THÁNG 5 NĂM 2026

KHOA ĐIỆN – ĐIỆN TỬ		NHIỆM VỤ KHÓA LUẬN TỐT NGHIỆP			
(Chú ý: sinh viên phải dán tờ này vào trang thứ nhất của bản thuyết minh)					
HỌ VÀ TÊN:	Nguyễn Xuân Phát	MSSV:	2212529	LỚP:	DD22DVT3
NGÀNH:	Điện tử – Viễn thông				
1.	Đầu đề khóa luận:				
	Thiết kế hệ thống giám sát năng lượng điện				
	(Design and Implementation of an Electrical Energy Monitoring System)				
2.	Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):				
-	Thiết kế và thi công phần cứng cho hệ thống.				
-	Thiết kế và phát triển phần mềm nhúng cho hệ thống.				
-	Thiết kế App nhập liệu các thông số về thiết bị thu thập dữ liệu.				
-	Phát triển trang Web theo dõi cho hệ thống.				
-					
3.	Ngày giao nhiệm vụ khóa luận: 05 / 01 / 2026				
4.	Ngày hoàn thành nhiệm vụ: DD / MM / YYYY				
5.	Họ và tên người hướng dẫn:		Phần hướng dẫn:		
-	ThS. Bùi Quốc Bảo		Thiết kế phần cứng Thiết kế phần mềm nhúng Phát triển giao diện Web		
Nội dung và yêu cầu KLTN đã được thông qua Chủ trì ngành.					

Ngày tháng năm

CHỦ TRÌ NGÀNH

(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH

(Ký và ghi rõ họ tên)

PHẦN DÀNH CHO KHOA:

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ luận án:

Lời cảm ơn

Để hoàn thành đồ án này, trước hết em xin gửi lời cảm ơn sâu sắc đến tập thể phòng Lab 209B3 và thầy Bùi Quốc Bảo, người đã trực tiếp hướng dẫn em trong luận văn tốt nghiệp này, cũng như thầy, cô giảng viên trong Khoa Điện – Điện tử, Trường Đại học Bách Khoa – ĐH Quốc gia TP. HCM, đã truyền đạt kiến thức và kinh nghiệm quý báu cho em trong suốt quá trình học tập và rèn luyện của em tại trường. Trong quá trình làm luận văn, em đã gặp không ít khó khăn, còn thiếu sót nhiều về mặt kiến thức thực tế. Nhưng với sự giúp đỡ, hỗ trợ từ anh Cao Văn Hiếu, anh Huỳnh Thanh Sang trong việc thiết kế phần cứng và những người bạn tại phòng Lab 209B3 trong việc chia sẻ kiến thức với em, và đặc biệt là sự giúp đỡ, tư vấn nhiệt tình từ thầy Bùi Quốc Bảo chia sẻ những kiến thức và kinh nghiệm của thầy để em có thể hoàn thành tốt luận văn của mình và tích lũy thêm cho bản thân được nhiều những kinh nghiệm thực tế, kiến thức hữu ích cho bản thân, cải thiện kỹ năng tìm kiếm thông tin trong quá trình thực tập để hoàn thành nhiệm vụ được thầy giao. Dù bản thân đã rất cố gắng để hoàn thành tốt các nhiệm vụ được giao nhưng không thể tránh khỏi những sai sót nên em rất mong nhận được sự thông cảm, đóng góp ý kiến của quý thầy cô.

Cuối cùng, em xin kính chúc quý thầy cô sức khỏe, luôn thành công trong công việc và cuộc sống. Em xin chân thành cảm ơn!

TP.Hồ Chí Minh, ngày 15 tháng 5 năm 2026

Nguyễn Xuân Phát

TÓM TẮT ĐỒ ÁN

Trong bối cảnh hiện nay, các doanh nghiệp đang có xu hướng công nghiệp hóa hiện đại hóa các quy trình đóng gói sản xuất, các thiết bị hiện đại có khả năng tự vận hành cũng vì thế mà phát triển ngày càng nhiều theo nhu cầu của thị trường. Các loại máy móc này hoạt động với tần suất liên tục suốt hàng giờ vì thế không tránh khỏi các tình trạng sai số do các linh kiện tạo ra, hỏng hóc trong quá trình vận hành do các yếu tố khách quan cũng như là chủ quan. Để có thể đảm bảo được quy trình sản xuất diễn ra đúng với kế hoạch của công ty thì công tác vận hành và bảo trì các loại máy móc cỡ lớn này cần được thực hiện thường xuyên để có thể kịp thời phát hiện ra lỗi và sửa chữa. Từ đó, các thông số về điện áp hoặc dòng điện sai lệch như thế nào so với các thông số ban đầu mà nhà sản xuất máy móc đưa ra sẽ được thu thập thông qua các đồng hồ đo điện năng (Power Meter) và ghi chép lại sau đó được tính toán một cách cẩn thận và đưa ra hướng giải quyết phù hợp nhất.

Đồ án này tập trung vào thiết kế một Gateway có chức năng thu thập dữ liệu từ các đồng hồ điện thông qua chuẩn Modbus trong công nghiệp, gửi dữ liệu về Cloud theo dõi định kỳ theo thời gian thực, tương tự với cách thức các doanh nghiệp thực hiện. Thiết bị sử dụng vi điều khiển tích hợp khả năng kết nối Wifi, Bluetooth (BLE) có thể nhận cấu hình thông qua Bluetooth từ App, chức năng chính của App này là nhập mới một thiết bị khi được thêm vào hệ thống, dữ liệu mới được thêm vào mà các thanh ghi lưu trữ dữ liệu bên trong đồng hồ điện. Dữ liệu sau khi được thiết bị thu thập sẽ được lưu tại thiết bị thông qua thẻ nhớ SD Card trong trường hợp thiết bị mất kết nối Internet không thể gửi dữ liệu về Cloud. Khi có kết nối Internet thông qua Wifi hoặc Ethernet được tích hợp trên thiết bị thì dữ liệu sẽ được gửi về Cloud thông qua giao thức MQTT, Cloud ở đây là một trang web được phát triển trên hệ điều hành Linux giúp người dùng có thể nhìn một cách trực quan hơn về dữ liệu và thiết bị thu thập được.

MỤC LỤC

1	Giới thiệu	1
1.1	Tổng quan về đề tài	1
1.2	Tình hình nghiên cứu trong và ngoài nước	1
1.3	Nhiệm vụ đề tài	2
2	Cơ sở lý thuyết	3
2.1	Tìm hiểu về lý thuyết về giao thức Modbus	3
2.1.1	Khái quát về giao thức Modbus	3
2.1.2	Tìm hiểu lý thuyết về giao thức Modbus RTU	4
2.1.3	Tìm hiểu Function Codes trong giao thức Modbus	7
2.2	Tìm hiểu về giao thức Ethernet	8
2.2.1	Khái quát và các đặc của giao thức Ethernet	8
2.2.2	Tìm hiểu về cấu trúc gói tin Ethernet	9
2.2.3	Cách thức đóng gói gói tin Ethernet theo mô hình OSI	10
2.3	Tìm hiểu về giao thức truyền thông MQTT	12
2.4	Tìm hiểu về giao thức Bluetooth Low Energy (BLE)	13
2.5	Tìm hiểu về các kiến thức liên quan tới trang web	14
2.6	Khái niệm về API RESTful và API Websocket	15
2.6.1	Khái niệm và đặc điểm về API RESTful	15
2.6.2	Khái niệm và đặc điểm về API Websocket	16
3	Thiết kế và thực hiện phần cứng	18
3.1	Thiết kế hệ thống	18
3.1.1	Tổng quan kiến trúc hệ thống	18
3.1.2	Các khối chức năng chính của trong hệ thống	20
3.1.3	Chi tiết thiết kế phần cứng	22
4	Thiết kế và thực hiện phần mềm	33
4.1	Thiết kế phần mềm cho thiết bị thu thập dữ liệu	33
4.1.1	Yêu cầu đặt ra cho phần mềm	33
4.1.2	Phương pháp thực hiện chương trình	33
4.1.3	Lưu đồ giải thuật chương trình	35

4.1.4	Lưu đồ giải thuật chi tiết	38
4.2	Thiết kế phần mềm App nhập dữ liệu	47
4.2.1	Thiết kế phần mềm	47
4.3	Phát triển giao diện Dashboard theo dõi dữ liệu	52
4.3.1	Thiết kế phần mềm	52
5	Kết quả thu được	55
5.1	Kết quả thực hiện phần cứng thực tế	55
5.1.1	Kết quả thiết kế phần cứng trên phần mềm	55
5.1.2	Thực hiện thi công mạch thực tế	57
5.2	Kết quả phần mềm App nhập dữ liệu	58
5.3	Kết quả phát triển giao diện Dashboard	60
6	Kết luận và hướng phát triển	62
6.1	Kết luận	62
6.2	Hướng phát triển	62

DANH SÁCH HÌNH VẼ

2-1	Tổng quan về giao thức Modbus	3
2-2	Tổng quan về giao thức Modbus RTU	4
2-3	Sơ đồ kết nối RS485 với điện trở đầu cuối 120Ω	5
2-4	Khung truyền tổng quát của Modbus RTU	5
2-5	Khung truyền Master gửi Request cho Slave trong Modbus RTU . . .	6
2-6	Khung truyền Slave gửi Response cho Master trong Modbus RTU . .	6
2-7	Khung truyền Slave gửi báo lỗi cho Master trong Modbus RTU	6
2-8	Tổng quan về mạng Ethernet	8
2-9	Cấu trúc của gói tin Ethernet	9
2-10	Cấu trúc tổng quát của mô hình OSI	10
2-11	Quy trình đóng gói gói tin Ethernet	11
2-12	Mô hình giao tiếp MQTT	12
2-13	Giao thức Bluetooth Low Energy (BLE)	13
2-14	Tổng quan về các thành phần của một trang web	14
2-15	Tổng quan về RESTful API	15
2-16	Tổng quan về WebSocket API	17
3-1	Sơ đồ khối tổng quát thực hiện phần cứng	18
3-2	Sơ đồ khối tổng quát của phần cứng	20
3-3	Sơ đồ khối phân bố nguồn trong hệ thống	21
3-4	khối vi điều khiển	22
3-5	khối nạp code và Debug	23
3-6	khối kiểm tra nguồn cho vi điều khiển	23
3-7	khối lọc nguồn đầu vào cho vi điều khiển	23
3-8	khối nguồn đầu vào	24
3-9	Khối nguồn hạ áp 5V	24
3-10	Khối nguồn hạ áp 3.3V	24
3-11	khối nguồn cách ly 5V	25
3-12	khối nguồn Pin rời cho mạch RTC	25
3-13	khối vi điều khiển cho Ethernet	26
3-14	khối cổng kết nối cho Ethernet	26
3-15	khối lọc nguồn đầu vào cho Ethernet	27
3-16	khối thạch anh và DIP Switch cho Ethernet	27

3-17 khối xử lý tín hiệu RS485	28
3-18 khối cách ly RS485	29
3-19 khối cổng giao tiếp RS485	29
3-20 khối lọc nguồn đầu vào và LED báo nguồn	29
3-21 khối bộ nhớ EEPROM	30
3-22 khối RTC	30
3-23 khối lọc nguồn đầu vào cho EEPROM và RTC	31
3-24 khối nút nhấn tương tác với người dùng	31
3-25 màn hình LCD hiển thị	32
4-1 Tổng quát kiến trúc phần mềm của thiết bị	35
4-2 Lưu đồ giải thuật thực hiện tổng quát của hệ thống	36
4-3 Lưu đồ giải thuật cấu hình Wi-Fi	38
4-4 Lưu đồ giải thuật cấu hình Ethernet	39
4-5 Lưu đồ giải thuật đợi nhận data từ App qua BLE	40
4-6 Lưu đồ giải thuật thu thập dữ liệu từ đồng hồ điện	41
4-7 Lưu đồ giải thuật thực hiện thay đổi tốc độ baudrate	42
4-8 Lưu đồ giải thuật chức năng scan đường dây	43
4-9 State machine mô tả chuyển trang của LCD	45
4-10 State machine mô tả trang Menu settings	45
4-11 Lưu đồ giải thuật mô tả hoạt động của trang Device info	46
4-12 Sơ đồ khối tổng quát của phần mềm App	47
4-13 Lưu đồ giải thuật chi tiết cho giao diện App	48
4-14 Lưu đồ giải thuật phần khởi động App	48
4-15 Lưu đồ giải thuật phần tab Setup	49
4-16 Lưu đồ giải thuật phần chức năng Sync	51
4-17 Lưu đồ giải thuật phần tab History	52
4-18 Sơ đồ tổng quát các phần của trang web	52
4-19 Lưu đồ giải thuật phần nhận dữ liệu từ thiết bị - MQTT Worker	53
5-1 Hình ảnh mạch 2D sau khi đã đi dây ở lớp Top và lớp Bottom	55
5-2 phần đặt các ký hiệu trên bề mặt của lớp Top	55
5-3 Hình ảnh mạch 3D khi chưa đặt linh kiện của lớp Top và lớp Bottom	56
5-4 Hình ảnh mạch 3D khi đã đặt linh kiện của lớp Top và lớp Bottom	56

5-5	Các khối chức năng trên mạch sau khi sắp xếp	56
5-6	Mạch khi chưa hàn linh kiện	57
5-7	Mạch sau khi đã hàn các linh kiện lên mạch	57
5-8	Tab nhập liệu chính khi vừa khởi động App	58
5-9	Box nhập ID cho thiết bị mới	58
5-10	Thông báo lỗi khi ID đã tồn tại và thông báo lỗi khi chưa chọn thiết bị	58
5-11	Thông báo có cập nhật dữ liệu mới không	59
5-12	Tab lịch sử của App	59
5-13	Lưu giữ liệu vào thẻ nhớ SD Card và lưu dữ liệu theo từng ngày	59
5-14	Hiển thị các site có trong hệ thống của user	60
5-15	Dữ liệu các đồng hồ điện hiển thị tổng quát trên trình duyệt web	60
5-16	Dữ liệu từng thanh ghi của từng đồng hồ điện	60
5-17	Bảng tổng quát các cảnh báo đã gửi lên từ thiết bị	61
5-18	Hiển thị cụ thể thông tin đường truyền dựa trên Alarm gửi lên	61

DANH SÁCH BẢNG

2-1	Một số Function Codes phổ biến trong giao thức Modbus RTU	7
2-2	Các phương thức HTTP và hành động tương ứng	16

CHƯƠNG 1 GIỚI THIỆU

1.1 Tổng quan về đề tài

Trong kỷ nguyên Công nghiệp 4.0, việc giám sát và quản lý năng lượng điện hiệu quả đã trở thành yêu cầu cấp thiết đối với các doanh nghiệp nhằm tối ưu hóa chi phí vận hành và đảm bảo an ninh năng lượng. Lĩnh vực hệ thống nhúng và Internet of Things (IoT) đóng vai trò then chốt trong việc kết nối các thiết bị đo lường truyền thống với hạ tầng mạng hiện đại. Tuy nhiên, một thách thức lớn hiện nay là sự không đồng nhất về giao thức truyền thông giữa các thiết bị hiện trường (thường sử dụng thông qua chuẩn RS485) và các hệ thống quản lý trung tâm (thường sử dụng Ethernet/WiFi). Do đó, việc nghiên cứu và phát triển các bộ chuyển đổi giao thức (Gateway) có khả năng thu thập, xử lý và truyền tải dữ liệu lên các hệ thống quản lý từ xa theo thời gian thực và là một hướng nghiên cứu có tính ứng dụng cao và thực tiễn.

Mục tiêu chính của đề tài này là nghiên cứu giải pháp thiết kế một Gateway dựa trên vi điều khiển ESP32 để giám sát các thông số điện năng từ các đồng hồ đo công suất đa năng, cụ thể là dòng thiết bị Power Meter. Quá trình nghiên cứu tập trung vào việc nghiên cứu sử dụng các chuẩn giao thức phổ biến hiện nay trong công nghiệp để thu thập các thông số điện năng quan trọng như điện áp, dòng điện, công suất thực, công suất biểu kiến, năng lượng tiêu thụ,..., ngoài ra còn tập trung nghiên cứu cách thức đưa dữ liệu thu thập được lên các hệ thống quản lý từ xa thông qua giao thức MQTT và giao thức HTTP. Bên cạnh đó, đề tài cũng hướng đến việc tối ưu hóa giao diện người dùng cục bộ thông qua màn hình LCD và người dùng có thể tương tác với thiết bị thông qua các nút nhấn, giúp kỹ thuật viên có thể tương tác và cấu hình hệ thống trực tiếp mà không cần thiết bị ngoại vi phức tạp.

1.2 Tình hình nghiên cứu trong và ngoài nước

Trên thế giới, việc nghiên cứu các bộ chuyển đổi giao thức (Gateway) cho mạng truyền thông công nghiệp đã trải qua nhiều giai đoạn phát triển. Các nghiên cứu ban đầu tập trung vào việc tối ưu hóa tính toán cho các dòng vi điều khiển 8-bit hoặc 16-bit để thực hiện các ngăn xếp giao thức phức tạp. Hiện nay, xu hướng quốc tế đang tập trung vào các hệ thống nhúng hiệu năng cao có tích hợp kết nối không dây. Các tập đoàn lớn như Moxa, Advantech hay Schneider Electric đã cung cấp các bộ Gateway công nghiệp (ví dụ dòng MGate) với độ tin cậy cực cao, hỗ trợ đa giao thức từ Modbus

RTU sang Modbus TCP/IP hoặc MQTT. Tuy nhiên, các thiết bị này thường có giá thành rất cao và tính tùy biến về giao diện người dùng (HMI) cục bộ còn hạn chế.

Tại Việt Nam, lĩnh vực thiết kế thiết bị IoT và Gateway công nghiệp đang phát triển mạnh mẽ, đặc biệt là trong bối cảnh các nhà máy đang thực hiện chuyển đổi số. Nhiều nghiên cứu tập trung vào việc sử dụng các nền tảng vi điều khiển phổ biến như ESP32, STM32 để xây dựng các giải pháp Gateway tùy chỉnh với chi phí thấp. Nhiều doanh nghiệp trong nước đã bắt đầu nội địa hóa các bộ thu thập dữ liệu (Data Logger) cho hệ thống điện mặt trời và nhà máy sản xuất. Các nghiên cứu này thường tập trung vào việc tích hợp các giao thức truyền thông công nghiệp như Modbus RTU, Modbus TCP/IP, MQTT để thu thập dữ liệu từ các thiết bị đo lường và truyền tải lên các hệ thống quản lý từ xa. Tuy nhiên, vẫn còn nhiều thách thức về độ ổn định của kết nối không dây, bảo mật dữ liệu và tối ưu hóa giao diện người dùng cục bộ.

1.3 Nhiệm vụ đề tài

Nội dung 1: Tìm hiểu lý thuyết liên quan đến giao thức Modbus, giao thức Ethernet, vi điều khiển ESP32.

Nội dung 2: Tìm hiểu lý thuyết về cách sử dụng một trang Web để hiển thị các thông số theo dõi sau khi thu thập từ các thiết bị.

Nội dung 3: Thiết kế phần cứng cho Gateway sử dụng vi điều khiển ESP32.

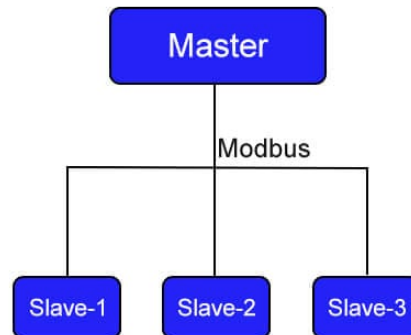
Nội dung 4: Phát triển phần mềm nhúng cho Gateway để thu thập dữ liệu từ các thiết bị đo lường thông qua giao thức Modbus RTU và truyền tải dữ liệu lên hệ thống quản lý từ xa thông qua giao thức MQTT.

Nội dung 5: Phát triển trang Web để hiển thị các thông số theo dõi sau khi thu thập từ các thiết bị.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Tìm hiểu về lý thuyết về giao thức Modbus

2.1.1 Khái quát về giao thức Modbus



Hình 2-1: Tổng quan về giao thức Modbus

Giao thức Modbus là một chuẩn giao thức truyền thông công nghiệp được phát triển bởi Modicon vào năm 1979. Là một trong những giao thức truyền nhận đơn giản đáng tin cậy, có khả năng tương thích cao trong các hệ thống điện. Modbus được sử dụng chủ yếu để trao đổi dữ liệu giữa các thiết bị điện tử công nghiệp với nhau, hoạt động theo mô hình Client-Server (Master-Slaves).

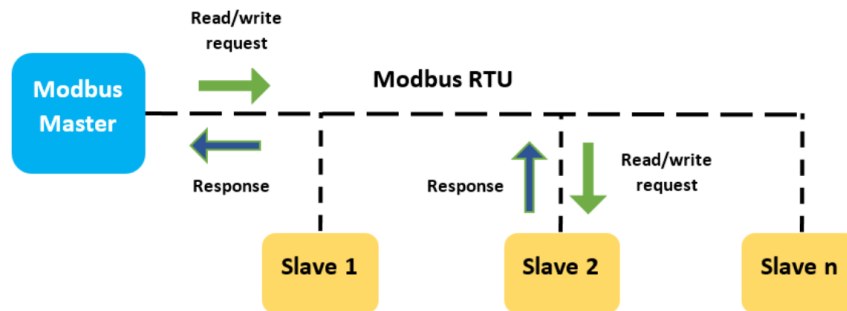
Giao thức được chia thành 3 biến thể thường được sử dụng trên thị trường, tùy thuộc vào môi trường làm việc sẽ sử dụng giao thức sao cho phù hợp, gồm 3 dạng chính sau:

- **Modbus RTU:** dữ liệu truyền dưới dạng số nhị phân, mô hình này sử dụng dạng truyền nhận có dây thường là các chuẩn như RS485, RS232 phù hợp với các mô hình giao tiếp nhỏ.
- **Modbus ASCII:** tương tự với Modbus RTU nhưng dữ liệu được truyền đi dưới dạng mã ASCII, có tốc độ chậm hơn Modbus RTU.
- **Modbus TCP/IP:** là hình thức truyền dữ liệu không dây theo chuẩn giao thức TCP/IP, sử dụng mạng Ethernet để truyền và nhận dữ liệu, thông dụng hơn trong

các hệ thống hiện đại.

2.1.2 Tìm hiểu lý thuyết về giao thức Modbus RTU

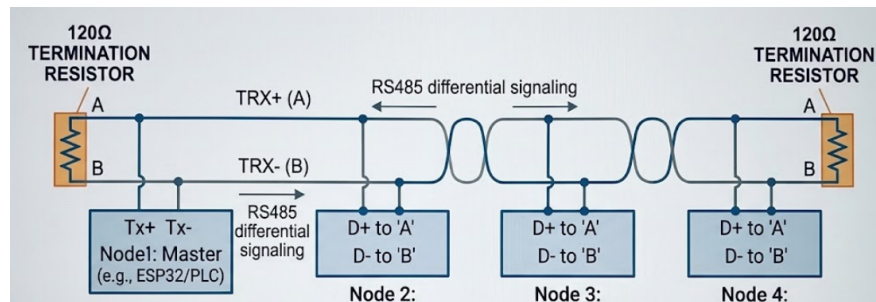
2.1.2.1 Tổng quan về giao thức Modbus RTU



Hình 2-2: Tổng quan về giao thức Modbus RTU

Giao thức Modbus RTU (Remote Terminal Unit) chạy trên nền tảng vật lý là RS485 (cho phép kết nối đa điểm với khoảng cách xa) hoặc RS232. Một thiết bị Master có thể kết nối và truy vấn tối đa lên đến 247 thiết bị Slave (mỗi Slave có một địa chỉ ID duy nhất từ 1 đến 247). Master sẽ gửi một yêu cầu (Request) chứa địa chỉ Slave, mã hàm (Function Code), địa chỉ thanh ghi và dữ liệu cần xử lý. Slave tương ứng sẽ nhận lệnh, thực hiện và phản hồi (Response) lại kết quả hoặc báo lỗi. Điểm đặc trưng của Modbus RTU là sử dụng phương pháp kiểm tra lỗi CRC (Cyclic Redundancy Check) 16-bit ở cuối mỗi khung truyền. Điều này đảm bảo dữ liệu không bị sai lệch do nhiễu gây ra.

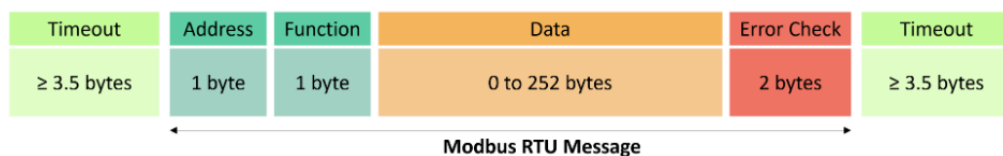
Tín hiệu dữ liệu được xác định bằng sự chênh lệch điện áp giữa dây A và dây B. Khi có nhiễu điện từ tác động vào đường dây, nó thường ảnh hưởng lên cả hai dây cùng lúc. Nhờ bộ thu vi sai, thiết bị sẽ triệt tiêu nhiễu này, giúp tín hiệu vẫn giữ được độ chính xác ngay cả trong môi trường có nhiều motor hoặc biến tần.



Hình 2-3: Sơ đồ kết nối RS485 với điện trở đầu cuối 120Ω

Trong mạng RS485, việc lắp đặt điện trở đầu cuối (thường có giá trị 120Ω) tại hai điểm đầu và cuối của đường bus là cực kỳ quan trọng. Điện trở này có nhiệm vụ phối hợp trở kháng của đường truyền. Khi tín hiệu điện di chuyển đến cuối đường dây, nếu không có điện trở này để "hấp thụ", tín hiệu sẽ bị phản xạ ngược lại (Signal Reflection).

2.1.2.2 Tìm hiểu khung truyền dữ liệu Modbus RTU



Hình 2-4: Khung truyền tổng quát của Modbus RTU

Một khung truyền Modbus RTU tiêu chuẩn bao gồm 4 thành phần chính được truyền dưới dạng các byte nhị phân liên tiếp. Tổng độ dài của một khung không được vượt quá 256 byte. Cấu trúc chi tiết như sau:

- **Địa chỉ thiết bị (Slave Address - 1 byte):** Xác định thiết bị đích nhận lệnh (từ 1 đến 247). Địa chỉ 0 được dùng cho chế độ Broadcast (gửi cho tất cả các thiết bị).
- **Mã hàm (Function Code- 1 byte):** Cho biết yêu cầu mà Master muốn Slave thực hiện
- **Dữ liệu (Data - 0-252 byte):** Chứa thông tin bổ sung tùy theo mã hàm, bao gồm địa chỉ thanh ghi, số lượng thanh ghi cần đọc hoặc giá trị cần ghi vào thiết bị.
- **Kiểm tra lỗi (CRC - 2 byte):** Sử dụng thuật toán kiểm tra dư thừa vòng (Cyclic Redundancy Check) 16-bit để đảm bảo tính toàn vẹn của dữ liệu trong quá trình truyền dẫn.

Quy định về thời gian (Timing): Trong chuẩn RTU, các byte trong một khung phải được truyền liên tục. Một khung truyền được xác định là kết thúc khi có một khoảng lặng (silence) trên đường truyền tối thiểu bằng thời gian truyền 3.5 ký tự (3.5 character time). Nếu khoảng cách giữa hai byte vượt quá 1.5 ký tự, thiết bị nhận sẽ coi đó là một lỗi và hủy bỏ khung truyền đó.

Slave Address (1 Byte)	Function Code (1 Byte)	Start Address (2 Bytes)	Quantity (2 Bytes)	CRC (2 Bytes - CRC bit)
---------------------------	---------------------------	----------------------------	-----------------------	----------------------------

Hình 2-5: Khung truyền Master gửi Request cho Slave trong Modbus RTU

Khung truyền yêu cầu (Master Request): được khởi xướng bởi thiết bị Master để yêu cầu dữ liệu hoặc ra lệnh cho một Slave cụ thể. Ngoài Slave Address và Function Code, khung truyền này chứa thông tin định hướng gồm Start Address (địa chỉ thanh ghi bắt đầu) và Quantity (số lượng thanh ghi cần thao tác). Cả hai trường này đều có độ dài 2 Byte, cho phép Master truy cập chính xác vào các vùng nhớ dữ liệu bên trong thiết bị Slave. Kết thúc khung truyền là mã CRC 16-bit để đảm bảo lệnh gửi đi không bị nhiễu làm sai lệch.

Slave Address (1 Byte)	Function Code (1 Byte)	Data (N Bytes)	CRC (2 Bytes - CRC bit)
---------------------------	---------------------------	-------------------	----------------------------

Hình 2-6: Khung truyền Slave gửi Response cho Master trong Modbus RTU

Khung truyền phản hồi bình thường (Slave Response): sau khi nhận được lệnh hợp lệ, thiết bị Slave sẽ gửi lại khung truyền Response để xác nhận hoặc trả về dữ liệu. Cấu trúc khung này lặp lại Slave Address và Function Code từ yêu cầu của Master để xác nhận đối tượng đang giao tiếp. Phần quan trọng nhất là trường Data (có độ dài N Bytes tùy thuộc vào lượng dữ liệu yêu cầu), chứa các giá trị thực tế từ thanh ghi hoặc trạng thái của Slave. Khung truyền này giúp Master cập nhật được thông tin từ các cảm biến hoặc thiết bị ngoại vi trong hệ thống.

Slave Address (1 Byte)	Error Function Code (1 Byte)	Exception Code (1 Byte)
---------------------------	---------------------------------	----------------------------

Hình 2-7: Khung truyền Slave gửi báo lỗi cho Master trong Modbus RTU

Khung truyền báo lỗi (Exception Response): trong trường hợp Slave nhận được yêu cầu nhưng không thể thực hiện (ví dụ: sai địa chỉ thanh ghi hoặc mã hàm không hỗ trợ), nó sẽ gửi một khung phản hồi ngoại lệ. Điểm khác biệt nằm ở Error Function Code, giá trị này bằng mã hàm gốc cộng thêm 0x80 (set bit MSB lên 1) để Master nhận diện đây là một thông báo lỗi. Đi kèm là Exception Code (1 Byte) dùng để mã hóa nguyên nhân cụ thể dẫn đến lỗi (như Illegal Data Address hoặc Illegal Function), giúp người vận hành dễ dàng chẩn đoán sự cố trong mạng truyền thông.

2.1.3 Tìm hiểu Function Codes trong giao thức Modbus

Function Codes	Description
0x01	Read Coils
0x02	Read Discrete Inputs
0x03	Read Holding Registers
0x04	Read Input Registers
0x05	Write Single Coils
0x06	Write Single Holding Register
0x0F	Write Multiple Coils
0x10	Write Multiple Holding Register

Bảng 2-1: Một số Function Codes phổ biến trong giao thức Modbus RTU

Function Code (mã lệnh) là thành phần quyết định loại thao tác dữ liệu mà Master yêu cầu Slave thực hiện, Được sử dụng cho cả Modbus RTU và Modbus TCP Các mã hàm phổ biến được phân chia dựa trên kiểu dữ liệu (Bit hoặc Word) và quyền truy cập (Đọc hoặc Ghi):

- **Nhóm đọc trạng thái Logic (0x01 và 0x02):** Mã hàm 0x01 (Read Coils) được sử dụng để đọc trạng thái ON/OFF của các đầu ra số như Relay hoặc đèn báo. Trong khi đó, mã hàm 0x02 (Read Discrete Inputs) dùng để kiểm tra trạng thái của các đầu vào số vật lý như nút nhấn hoặc công tắc hành trình, vốn là các dữ liệu chỉ cho phép đọc.
- **Nhóm đọc dữ liệu thanh ghi (0x03 và 0x04):** Để lấy dữ liệu 16-bit, mã hàm 0x03 (Read Holding Registers) thường được dùng để đọc các thông số cấu hình hoặc giá trị đo lường có khả năng thay đổi. Ngược lại, mã hàm 0x04 (Read Input Registers) chuyên dùng để đọc các giá trị từ cảm biến hoặc dữ liệu thô từ phần cứng mà Master không có quyền ghi đè.
- **Nhóm ghi dữ liệu đơn lẻ (0x05 và 0x06):** Khi cần điều khiển một đối tượng duy

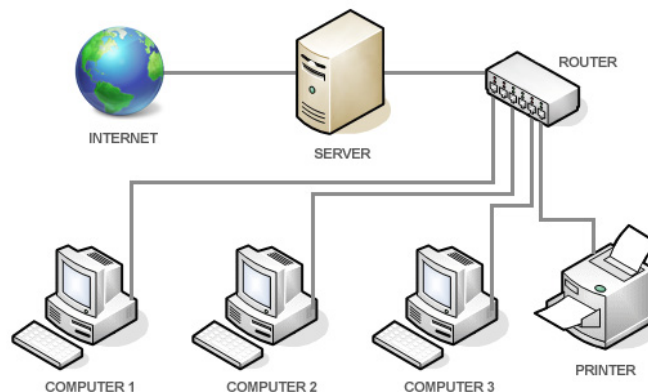
nhất, Master sử dụng mã hàm 0x05 (Write Single Coil) để đóng/ngắt một Relay hoặc mã hàm 0x06 (Write Single Register) để thay đổi giá trị của một thanh ghi cấu hình cụ thể (như cài đặt tần số cho biến tần).

- **Nhóm ghi dữ liệu đa nhiệm (0x0F và 0x10):** Để tối ưu hóa tốc độ truyền thông, mã hàm 0x0F (Write Multiple Coils) và 0x10 (Write Multiple Registers) cho phép Master ghi đồng thời một nhóm nhiều trạng thái Bit hoặc nhiều thanh ghi 16-bit liên tiếp chỉ trong một khung truyền duy nhất. Điều này đặc biệt hữu ích khi cần thiết lập đồng loạt nhiều tham số cho hệ thống.

2.2 Tìm hiểu về giao thức Ethernet

2.2.1 Khái quát và các đặc của giao thức Ethernet

Ethernet là giao thức truyền thông phổ biến nhất trong các mạng cục bộ (LAN), hoạt động chủ yếu ở tầng Vật lý (Physical) và tầng Liên kết dữ liệu (Data Link) trong mô hình 7 lớp OSI. Được chuẩn hóa theo chuẩn IEEE 802.3, Ethernet cho phép các thiết bị trong mạng chia sẻ dữ liệu thông qua các gói tin (frames) với tốc độ cao, từ 10 Mbps đến hàng trăm Gbps.



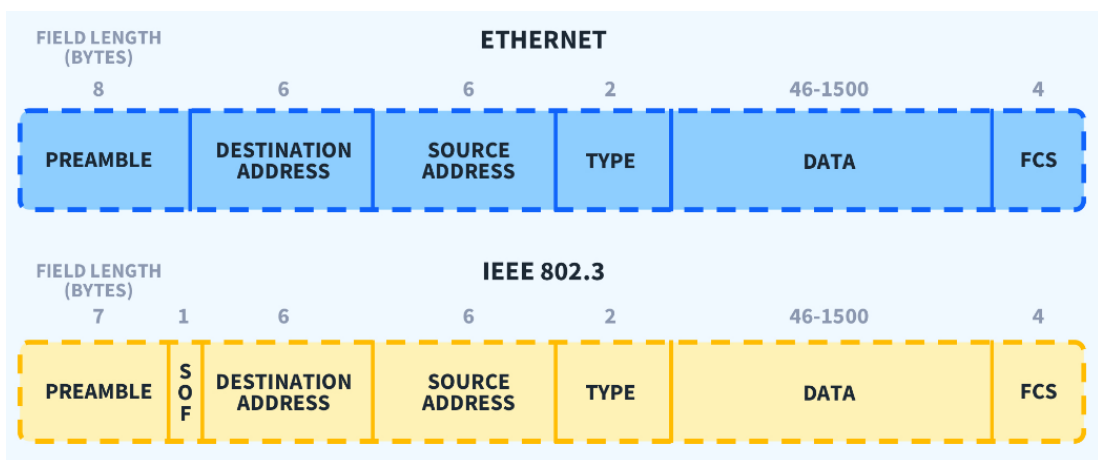
Hình 2-8: Tổng quan về mạng Ethernet

Các đặc trưng của giao thức Ethernet:

- **Cơ chế truy cập đường truyền (CSMA/CD):** Trước khi gửi dữ liệu, thiết bị sẽ "nghe" đường truyền. Nếu trống, nó mới gửi tin. Nếu có hai thiết bị gửi cùng lúc (xung đột), chúng sẽ dừng lại, chờ một khoảng thời gian ngẫu nhiên rồi mới thử lại. Trong các mạng hiện đại dùng Switch (Full-duplex), xung đột này hầu như đã được triệt tiêu.

- **Địa chỉ vật lý (MAC Address):** Mỗi thiết bị sử dụng mạng Ethernet để truyền dữ liệu đều có một địa chỉ MAC duy nhất gồm 48-bit (6 byte). Đây là định danh duy nhất để các thiết bị nhận diện nhau trong cùng một phân đoạn mạng.
- **Cấu trúc Frame:** Một gói tin Ethernet bao gồm các trường quan trọng như: Preamble (Đồng bộ hóa), Địa chỉ MAC nguồn/đích, Loại giao thức (EtherType), Dữ liệu (Payload) và mã kiểm tra lỗi FCS (Frame Check Sequence).
- **Chuẩn dây vật lý:** Sử dụng cáp xoắn đôi (Twisted Pair) như Cat5e, Cat6 với đầu nối RJ45 hoặc M12, giúp triệt tiêu nhiễu điện từ tương tự như nguyên lý vi sai của RS485 nhưng ở băng thông lớn hơn rất nhiều.
- **Tính đáp ứng thời gian thực:** Ethernet trong công nghiệp được cải tiến để chịu được môi trường khắc nghiệt (nhiều điện từ, nhiệt độ cao). Các giao thức như Modbus TCP/IP chạy trên nền Ethernet đảm bảo dữ liệu từ cảm biến được gửi về Gateway với độ trễ cực thấp.

2.2.2 Tìm hiểu về cấu trúc gói tin Ethernet



Hình 2-9: Cấu trúc của gói tin Ethernet

IEEE 802.3 là một tiêu chuẩn do tổ chức IEEE ban hành nhằm chuẩn hóa quy tắc chung cho công nghệ Ethernet, tập hợp các chuẩn của IEEE định nghĩa lớp vật lý và lớp con MAC của tầng liên kết dữ liệu trong mạng Ethernet có dây.

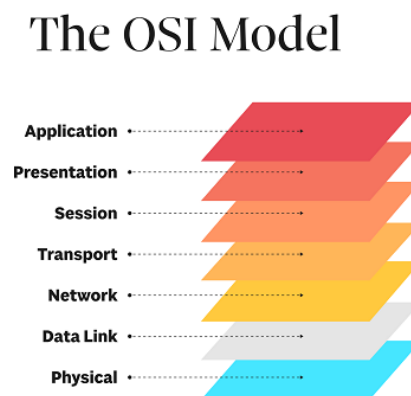
Các thành phần của một gói tin Ethernet như sau:

- **Preamble (Ethernet (8 bytes) / IEEE 802.3 (7 bytes)):** Là một dãy các bit 0 và 1 xen kẽ nhau. Có chức năng giúp thiết bị nhận đồng bộ hóa xung nhịp (clock) với thiết bị gửi.

- **SOF (Start of Frame Delimiter - 1 byte):** Chỉ xuất hiện riêng biệt trong chuẩn IEEE 802.3, Ở Ethernet nó được tính gộp vào byte cuối của Preamble. có chức năng Đánh dấu điểm kết thúc của quá trình đồng bộ và báo hiệu rằng byte tiếp theo chính là địa chỉ đích.
- **Destination Address (6 bytes):** chứa địa chỉ MAC đích, thiết bị nhận sẽ nhìn vào đây, nếu MAC này trùng với MAC của nó (hoặc là địa chỉ Broadcast), nó sẽ tiếp tục đọc gói tin nếu không nó sẽ bỏ qua gói tin.
- **Source Address (6 bytes):** Chứa địa chỉ MAC nguồn (thiết bị gửi gói tin), giúp thiết bị nhận biết ai đã gửi dữ liệu để có thể phản hồi lại khi cần.
- **Type (2 bytes):** Xác định giao thức tầng mạng (Layer 3) đang nằm bên trong phần Data, sử dụng IPv4 hay IPv6.
- **Data (Payload) (46 - 1500 bytes):** Chứa dữ liệu người dùng muốn gửi đi.
- **FCS (Frame Check Sequence - 4 bytes):** Sử dụng thuật toán kiểm tra mã CRC. Thiết bị nhận sẽ tính toán lại mã này từ dữ liệu nhận được và so sánh với mã FCS ở cuối gói tin. Nếu không khớp, gói tin bị coi là lỗi và bị hủy bỏ ngay lập tức.

2.2.3 Cách thức đóng gói gói tin Ethernet theo mô hình OSI

2.2.3.1 Tìm hiểu về mô hình OSI

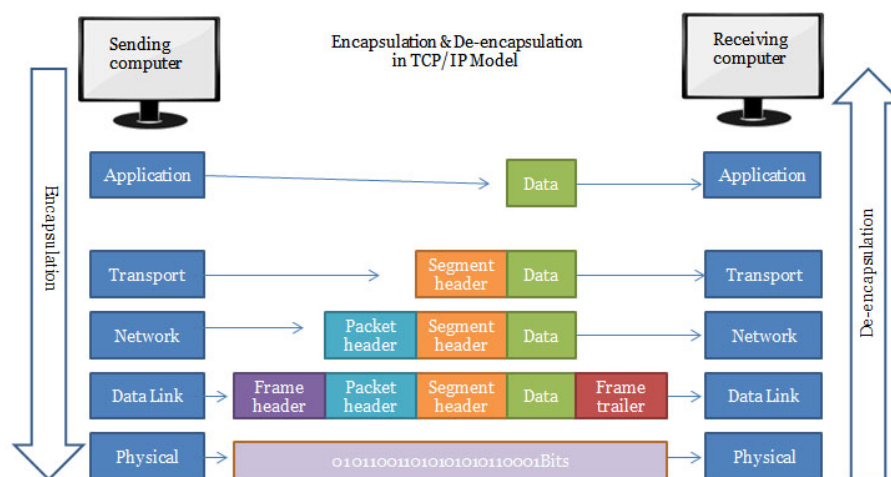


Hình 2-10: Cấu trúc tổng quát của mô hình OSI

Mô hình OSI (Open Systems Interconnection) là một khung lý thuyết được bộ tiêu chuẩn quốc tế (ISO) đưa ra để chuẩn hóa cách các hệ thống máy tính truyền thông với nhau Chức năng của từng thành phần trong mô hình:

- **Application Layer:** Giao diện trực tiếp với người dùng và các tiến trình phần mềm, nơi data chính trong gói tin được sinh ra, như HTTP (Web), MQTT (IoT), Modbus (Dữ liệu người dùng), Email (SMTP).
- **Presentation Layer:** lo việc định dạng dữ liệu, mã hóa (Encryption) và nén dữ liệu để đảm bảo máy nhận có thể đọc được.
- **Session Layer:** thiết lập, duy trì và đồng bộ hóa cuộc hội thoại giữa hai thiết bị. Nếu kết nối bị ngắt, tầng này lo việc khôi phục từ điểm kiểm soát.
- **Transport Layer:** Đảm bảo dữ liệu đi từ điểm A đến điểm B một cách toàn vẹn. Nó phân đoạn dữ liệu lớn thành các mảnh nhỏ và kiểm soát lưu lượng để tránh nghẽn mạng.
- **Network Layer:** Quyết định lộ trình tốt nhất để gói tin đi qua các mạng khác nhau dựa trên địa chỉ IP.
- **Data Link Layer:** Đóng gói dữ liệu thành các Frame, kiểm tra lỗi vật lý (CRC) và định địa chỉ vật lý (MAC).
- **Physical Layer:** Truyền dẫn dòng bit thô qua các phương tiện vật lý bằng tín hiệu điện, ánh sáng hoặc sóng vô tuyến.

2.2.3.2 Cách thức đóng gói tin Ethernet



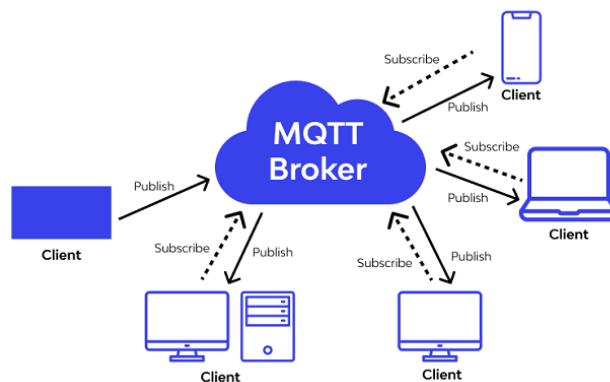
Hình 2-11: Quy trình đóng gói gói tin Ethernet

Các bước đóng gói gói tin Ethernet:

- **Application, Presentation, Session - Layer 7, 6, 5:** Chuyển đổi dữ liệu người dùng từ ứng dụng sang dạng nhị phân, mã hóa và thiết lập phiên truyền dẫn.

- **Transport - Layer 4:** Chia nhỏ dữ liệu (Segmentation) và thêm Transport Header, chịu trách nhiệm vận chuyển đầu-cuối. Thông tin quan trọng nhất ở đây là Số hiệu cổng (Port Number) để xác định ứng dụng nào đang gửi/nhận.
- **Network - Layer 3:** Bao bọc Segment bằng một Network Header. Thông tin cốt lõi là Địa chỉ Logic (như địa chỉ IP) của máy nguồn và máy đích.
- **Data Link - Layer 2:** Bao bọc Packet bằng một Data Link Header (chứa địa chỉ vật lý như MAC) và một Trailer (chứa mã kiểm tra lỗi FCS/CRC), chuẩn bị dữ liệu để truyền trên đường dây vật lý cụ thể.
- **Physical - Layer 1:** Chuyển toàn bộ Frame thành các xung điện, ánh sáng hoặc sóng vô tuyến.

2.3 Tìm hiểu về giao thức truyền thông MQTT



Hình 2-12: Mô hình giao tiếp MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông không dây, chạy trên nền tảng TCP/IP, có mã nguồn mở, được thiết kế cho các ứng dụng IoT với băng thông thấp, độ trễ cao, hoặc thiết bị có tài nguyên hạn chế. Là một phần trung gian cho phép các MQTT Client giao tiếp với nhau thông qua các chủ đề (Topic) đã đăng ký, có thể hiểu một cách đơn giản MQTT như một nhà phân phối sản phẩm theo nhu cầu của từng khách hàng thông qua Topic mà khách hàng đã đăng ký. Hoạt động dựa trên mô hình chính là Publish/Subscribe (Pus/Sub) và các thông tin được tổ chức theo Topic, các Slaves có thể gửi dữ liệu theo chủ đề để gửi đến những thiết bị Master (Subscriber) đã đăng ký nhận dữ liệu đó.

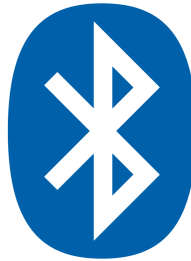
Trong MQTT có hai phần chính là MQTT Broker và MQTT Clients như sau:

- **MQTT Broker:** là nơi tiếp nhận các thông tin cần xử lý của các thiết bị trong hệ

thống, chịu trách nhiệm định tuyến hướng đi của các dữ liệu sao cho đến được chính xác vị trí mong muốn cần truyền giữa hai thiết bị đang muốn giao tiếp với nhau, có thể coi như là một nhà trung gian giữa hai thiết bị

- **MQTT Client:** là các thiết bị thu thập dữ liệu hay các thiết bị điều khiển hoặc cũng có thể là các ứng dụng, Web. Các thiết bị này sẽ không trực tiếp giao tiếp với nhau mà chúng sẽ thông qua MQTT Broker, hoạt động dựa trên thư viện MQTT và thiết lập kết nối với MQTT Broker.

2.4 Tìm hiểu về giao thức Bluetooth Low Energy (BLE)



Hình 2-13: Giao thức Bluetooth Low Energy (BLE)

BLE (Bluetooth Low Energy) là một phiên bản của Bluetooth được thiết kế tối ưu cho việc tiêu thụ điện năng cực thấp và truyền dữ liệu. Khác với giao thức Bluetooth truyền thống, BLE dành phần lớn thời gian ở chế độ "ngủ" và chỉ thức dậy trong vài mili giây để gửi các gói tin ngắn. BLE đã trở thành một lựa chọn phổ biến cho các thiết bị IoT (Internet of Things), thiết bị đeo thông minh, cảm biến y tế, và nhiều ứng dụng khác yêu cầu kết nối không dây với mức tiêu thụ năng lượng và chi phí thấp.

BLE hoạt động trong dải tần số 2.4 GHz ISM (Industrial, Scientific, and Medical), tương tự như Bluetooth truyền thống. Tuy nhiên, BLE sử dụng một phương pháp truyền dữ liệu khác biệt, tập trung vào việc gửi các gói dữ liệu nhỏ và ngắn trong khoảng thời gian ngắn, giúp giảm thiểu tiêu thụ năng lượng

Bluedroid là Bluetooth stack (ngăn xếp Bluetooth) mặc định của ESP32, được phát triển ban đầu bởi Broadcom và sau đó được Google tích hợp vào Android. ESP-IDF của Espressif sử dụng lại stack này để cung cấp đầy đủ chức năng Bluetooth Classic và BLE. Làm việc chủ yếu với 2 tầng trên cùng của Bluedroid:

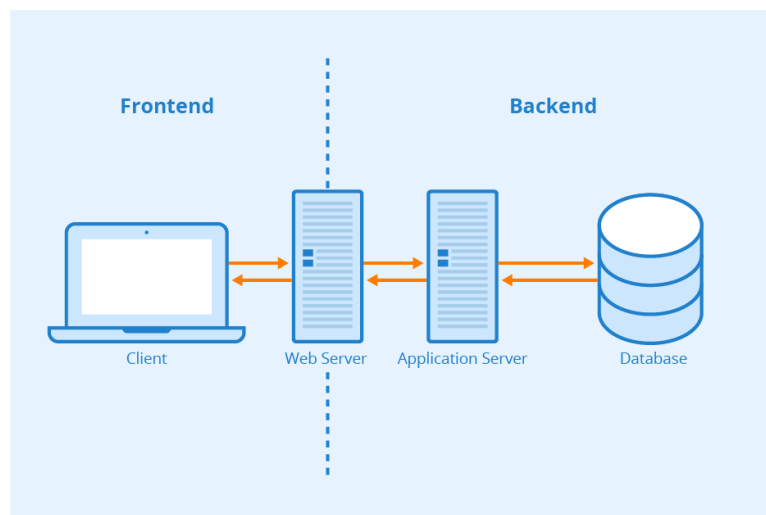
- **GAP (Generic Access Profile):** Quản lý việc quảng cáo (Advertising) và kết

nối. Nó quyết định việc ESP32 sẽ phát ra tín hiệu gì để các thiết bị khác có thể quét thấy, và thiết lập kết nối như thế nào.

- **GATT (Generic Attribute Profile):** Quản lý dữ liệu, tổ chức dữ liệu thành các bảng (Services) và đặc tính (Characteristics). GATT cho phép thiết bị khác đọc/ghi dữ liệu từ ESP32 thông qua các đặc tính đã được định nghĩa sẵn.
- **HCI (Host Controller Interface):** Đây là tầng cầu nối. Lệnh từ GAP hoặc GATT sẽ đi qua HCI để truyền xuống phần cứng phát ra ngoài không khí.

2.5 Tìm hiểu về các kiến thức liên quan tới trang web

Một ứng dụng web hiện đại là hệ thống phần mềm hoạt động theo mô hình Client-Server, được cấu thành từ ba trụ cột chính: Frontend (Giao diện), Backend (Hệ thống xử lý) và Database (Cơ sở dữ liệu). Trong đó, Frontend chịu trách nhiệm hiển thị nội dung và tương tác trực quan với người dùng thông qua các công nghệ cốt lõi như HTML, CSS và JavaScript. Phía sau giao diện là Backend, nơi chứa các logic nghiệp vụ (Business Logic), thuật toán xử lý và các API để điều phối dữ liệu. Cuối cùng, Database đóng vai trò là kho lưu trữ thông tin bền vững, quản lý việc truy xuất và cập nhật dữ liệu. Các thành phần này được liên kết chặt chẽ và vận hành trên hạ tầng Internet thông qua giao thức mạng HTTP/HTTPS, hệ thống tên miền (DNS) và máy chủ web (Web Server).



Hình 2-14: Tổng quan về các thành phần của một trang web

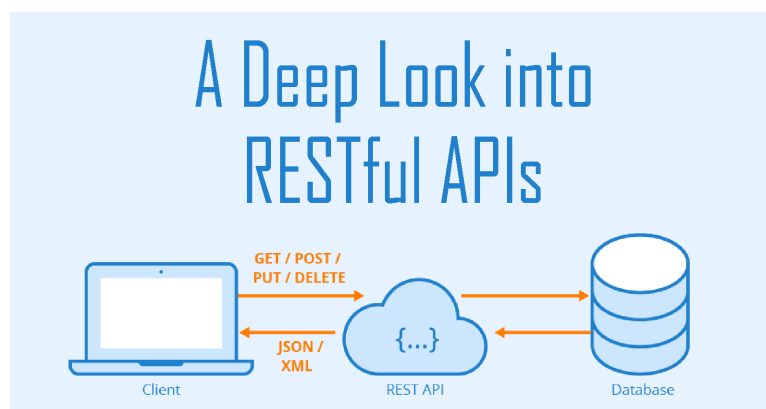
Các thành phần cốt lõi bao gồm:

- **Frontend (Client-side):** Là phần người dùng nhìn thấy và tương tác. Nó bao gồm cấu trúc trang (HTML), định dạng hiển thị (CSS) và các hiệu ứng/xử lý động (JavaScript/Frameworks như React, Vue).
- **Backend (Server-side):** Là phần chạy ngầm trên máy chủ, xử lý các yêu cầu từ Frontend. Nó bao gồm ngôn ngữ lập trình server (như Python, Node.js, PHP) và Web Server (như Nginx, Apache).
- **Database (Cơ sở dữ liệu):** Nơi lưu trữ dữ liệu người dùng, bài viết, sản phẩm... (thường là MySQL, PostgreSQL hoặc MongoDB).
- **Giao thức và Hạ tầng:** Bao gồm HTTP/HTTPS (luật truyền tải dữ liệu) và DNS (hệ thống phân giải tên miền giúp biến google.com thành địa chỉ IP máy chủ).

2.6 Khái niệm về API RESTful và API Websocket

2.6.1 Khái niệm và đặc điểm về API RESTful

RESTful API (Representational State Transfer) là một chuẩn kiến trúc phần mềm dùng để thiết kế các dịch vụ Web (Web Services), cho phép các ứng dụng trao đổi dữ liệu qua lại thông qua giao thức HTTP. Trong mô hình này, dữ liệu được coi là các "Tài nguyên" (Resources) và được định danh bằng các đường dẫn URI (Uniform Resource Identifier). Nguyên tắc cốt lõi của RESTful API là sự đơn giản, độc lập (Stateless) và sử dụng phương thức HTTP tiêu chuẩn để thực hiện các hành vi thao tác dữ liệu.



Hình 2-15: Tổng quan về RESTful API

V

Để một API được gọi là chuẩn RESTful, nó phải tuân thủ các nguyên tắc sau:

- **Stateless (Phi trạng thái):** Đây là đặc điểm quan trọng nhất. Server không lưu

Phương thức HTTP	Hành động tương ứng	Ý nghĩa	Ví dụ
GET	Read	Lấy dữ liệu từ server về.	GET /users (Lấy danh sách người dùng)
POST	Create	Gửi dữ liệu mới lên để tạo tài nguyên.	POST /users (Tạo người dùng mới)
PUT / PATCH	Update	Cập nhật thông tin của tài nguyên.	PUT /users/1 (Sửa thông tin user số 1)
DELETE	Delete	Xóa tài nguyên khỏi hệ thống.	DELETE /users/1 (Xóa user số 1)

Bảng 2-2: Các phương thức HTTP và hành động tương ứng

giữ trạng thái của Client giữa các lần gửi request. Mỗi request gửi lên phải chứa đầy đủ thông tin cần thiết (như token xác thực) để Server hiểu và xử lý.

- **Resource-based (Dựa trên tài nguyên):** Mọi thứ đều được xem là tài nguyên và được truy cập qua một đường dẫn (Endpoint) cụ thể. Ví dụ: /products, /orders.
- **Representation (Định dạng dữ liệu):** Dữ liệu trả về cho Client thường ở định dạng JSON (phổ biến nhất hiện nay vì nhẹ và dễ đọc) hoặc XML.

HTTP Status Codes (Mã phản hồi) sử dụng chuẩn code để báo kết quả, bao gồm một số mã sử dụng chủ yếu như sau:

- **200 OK:** Thành công.
- **201 Created:** Tạo mới thành công.
- **400 Bad Request:** Lỗi do gửi dữ liệu sai.
- **401 Unauthorized:** Chưa đăng nhập/không có quyền.
- **404 Not Found:** Không tìm thấy tài nguyên.
- **500 Internal Server Error:** Lỗi tại Server.

2.6.2 Khái niệm và đặc điểm về API Websocket

WebSocket là một giao thức truyền thông máy tính tiên tiến, cung cấp các kênh giao tiếp song phương (full-duplex) qua một kết nối TCP duy nhất. Khác với giao thức HTTP truyền thống (vốn hoạt động theo cơ chế 'hỏi - đáp' và ngắt kết nối ngay sau khi hoàn tất), WebSocket cho phép thiết lập một kết nối bền vững (persistent connection)

giữa Client và Server. Nhờ đó, dữ liệu có thể được truyền tải qua lại tức thì trong thời gian thực (Real-time) mà không cần Client phải gửi yêu cầu liên tục.



Hình 2-16: Tổng quan về Websocket API

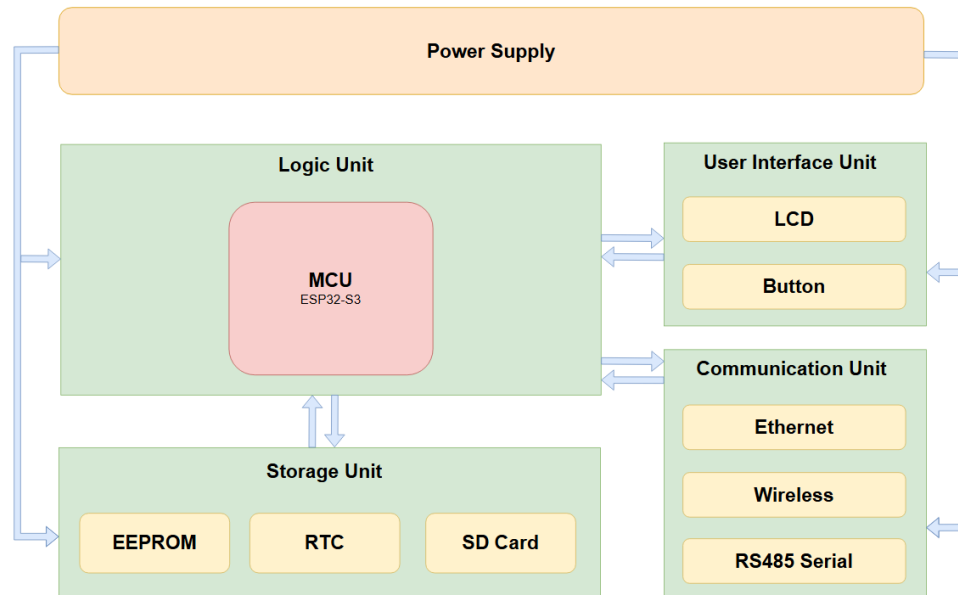
WebSocket hoạt động khác biệt hoàn toàn so với mô hình Request-Response của REST, quá trình này diễn ra qua 2 giai đoạn chính sau:

- **Giai đoạn Bắt tay (Handshake):** ban đầu, Client gửi một HTTP Request thông thường nhưng kèm theo header Upgrade: websocket. Nếu Server chấp nhận, nó sẽ phản hồi mã 101 Switching Protocols. Lúc này, giao thức HTTP được chuyển đổi (upgrade) sang giao thức WebSocket.
- **Giai đoạn Truyền dữ liệu (Data Transfer):** kết nối TCP được giữ nguyên (không đóng lại). Dữ liệu được đóng gói thành các Frame nhẹ và truyền qua lại hai chiều bất cứ lúc nào. Server chủ động đẩy (Push) dữ liệu xuống Client ngay khi có tin mới mà không cần Client phải "hỏi".

CHƯƠNG 3 THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG

3.1 Thiết kế hệ thống

3.1.1 Tổng quan kiến trúc hệ thống



Hình 3-1: Sơ đồ khối tổng quát thực hiện phần cứng

Khối nguồn (Power Supply): có vai trò hạ áp từ nguồn một chiều DC 12-24V xuống mức 5V và 3.3V để cung cấp điện cho các thành phần khác trong hệ thống, đảm bảo hoạt động ổn định và an toàn.

Khối vi điều khiển xử lý trong tâm (Logic Unit): sử dụng ESP32-S3 được cung cấp khả năng tích hợp WiFi và BLE Xử lý các tác vụ chính của hệ thống, đảm nhận các nhiệm vụ sau:

- Giao tiếp với các thiết bị ngoại vi như LCD, module RS485, RTC và EEPROM thông qua các giao thức SPI, I2C và UART.
- Thu thập dữ liệu từ các thiết bị giám sát (Inverter, Meter, Weather Station) qua giao thức Modbus RTU qua RS485.
- Giao tiếp với web hiển thị dữ liệu thông qua giao thức MQTT.

Khối lưu trữ (Storage Unit): tổ chức lưu trữ các dữ liệu như dữ liệu thu thập được, thông tin cấu hình wifi, cụ thể mỗi phần đảm nhận các nhiệm vụ sau:

- Khối RTC (Real-Time Clock): sử dụng module DS3231 để cung cấp thời gian

chính xác cho hệ thống, hỗ trợ việc đánh dấu thời gian cho dữ liệu thu thập và đồng bộ hóa với Server.

- Khối EEPROM: sử dụng module 24C256 để lưu trữ dữ liệu tạm thời khi mất kết nối mạng hoặc khi cần lưu trữ các thông số cấu hình của hệ thống như thông tin kết nối wifi,...
- Khối SD Card: được sử dụng để lưu trữ dữ liệu thu thập được từ các thiết bị kèm theo thời gian ghi nhận.

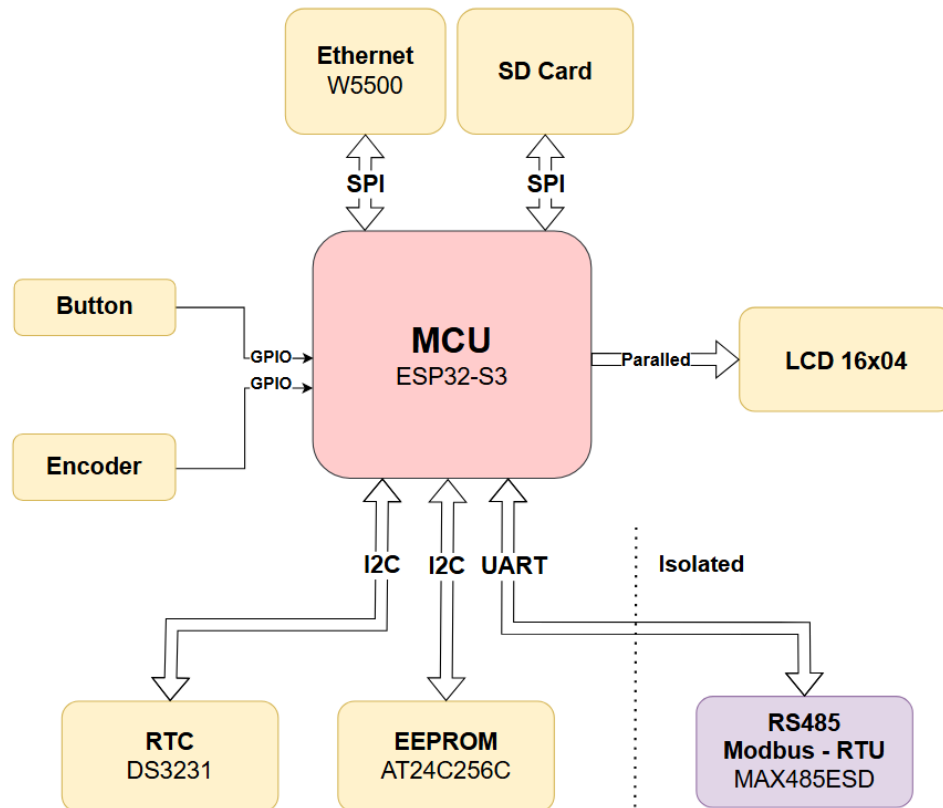
Khối giao tiếp (Communication Unit): bao gồm các giao thức truyền thông:

- Giao tiếp Modbus RTU qua RS485: để thu thập dữ liệu từ các thiết bị giám sát năng lượng điện Power Meter. Giao thức này được lựa chọn do tính ổn định và khả năng truyền dữ liệu ở khoảng cách xa trong môi trường công nghiệp.
- Giao tiếp BLE: Được sử dụng để nhận cấu hình từ App của người dùng gửi xuống.
- Giao tiếp MQTT: sử dụng kết nối wifi 2.4GHz hoặc thông qua cổng Ethernet để truyền dữ liệu định kỳ theo thời gian đến web hiển thị, giúp người dùng có thể theo dõi tình trạng hệ thống một cách trực quan và kịp thời.

Khối giao tiếp với người dùng (User Interface Unit) bao gồm các thành phần:

- LCD 16x04: hiển thị các thông số tổng quát về thiết bị.
- Nút nhấn và Encoder: để người dùng có thể tương tác, thay đổi thông số cấu hình cho thiết bị thông qua màn hình LCD

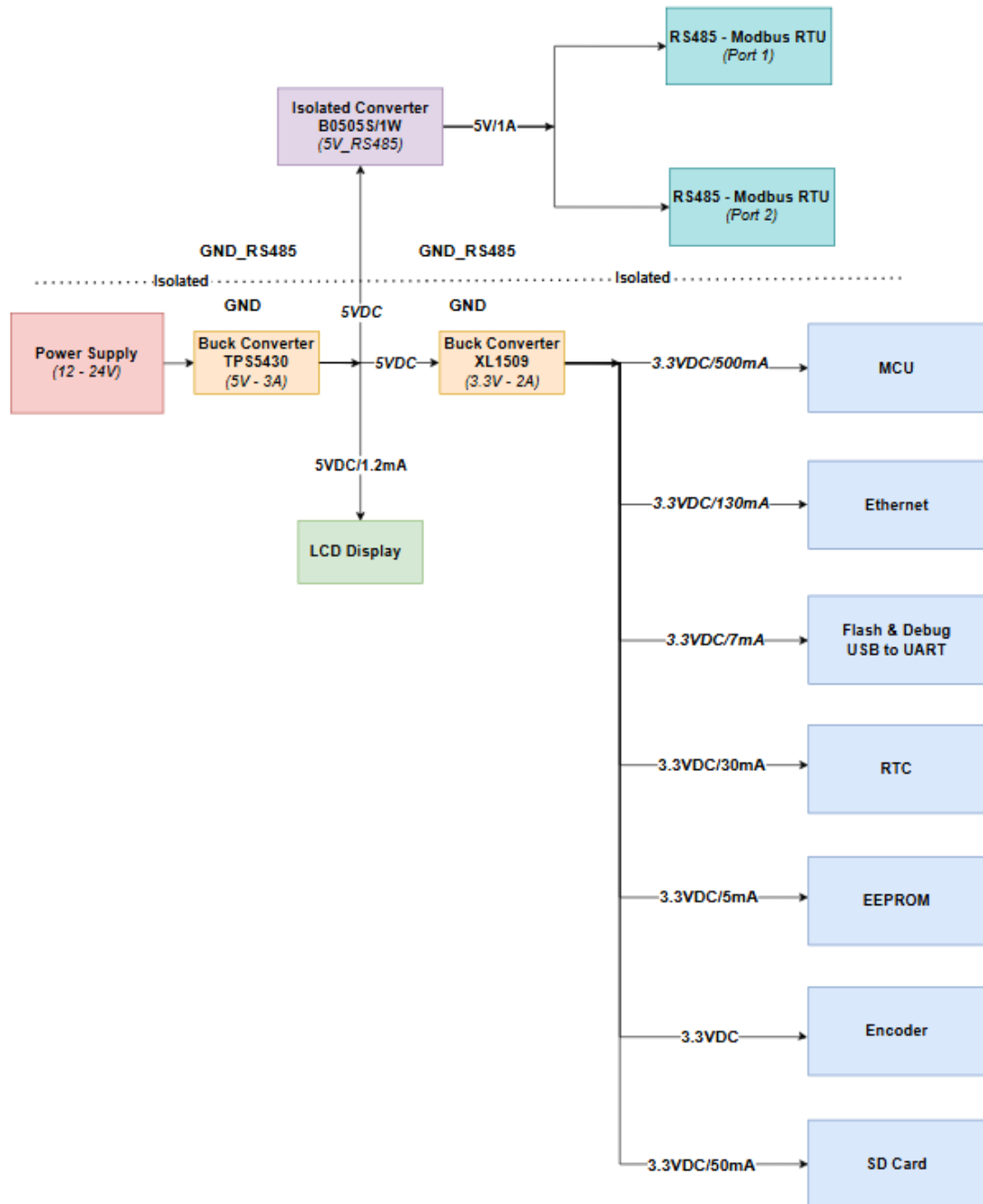
3.1.2 Các khối chức năng chính của trong hệ thống



Hình 3-2: Sơ đồ khối tổng quát của phần cứng

- Dữ liệu từ các thiết bị giám sát (các Power Meter) được thu thập định kỳ thông qua giao thức Modbus RTU qua chuẩn vật lý RS485 và được xử lý bởi ESP32.
- Dữ liệu sau khi được xử lý sẽ được truyền về Server thông qua giao thức MQTT qua Ethernet hoặc Wi-Fi. Đồng thời, dữ liệu về tình trạng đường truyền cũng được truyền đến web hiển thị thông qua giao thức MQTT để người dùng có thể theo dõi tình trạng hệ thống một cách trực quan và kịp thời.
- Các thông số tổng như tổng công suất, tổng điện áp, tổng dòng điện,... sẽ được hiển thị trên LCD 16x04 để người dùng có thể theo dõi nhanh tình trạng hệ thống ngay tại chỗ.
- Người dùng có thể tương tác với hệ thống thông qua màn hình LCD hiển thị các thông số và các nút nhấn tùy chỉnh.
- Các thông số cấu hình như wifi sẽ được lưu trữ ở EEPROM, thiết bị sau khi khởi động lại sẽ đọc các thông số cấu hình từ EEPROM để nạp lại cho hệ thống.

Cấu trúc Power Tree của hệ thống được thiết kế để đảm bảo nguồn điện được phân phối một cách hiệu quả và ổn định đến tất cả các thành phần, đồng thời cung cấp khả năng mở rộng trong tương lai nếu cần thiết.



Hình 3-3: Sơ đồ khối phân bố nguồn trong hệ thống

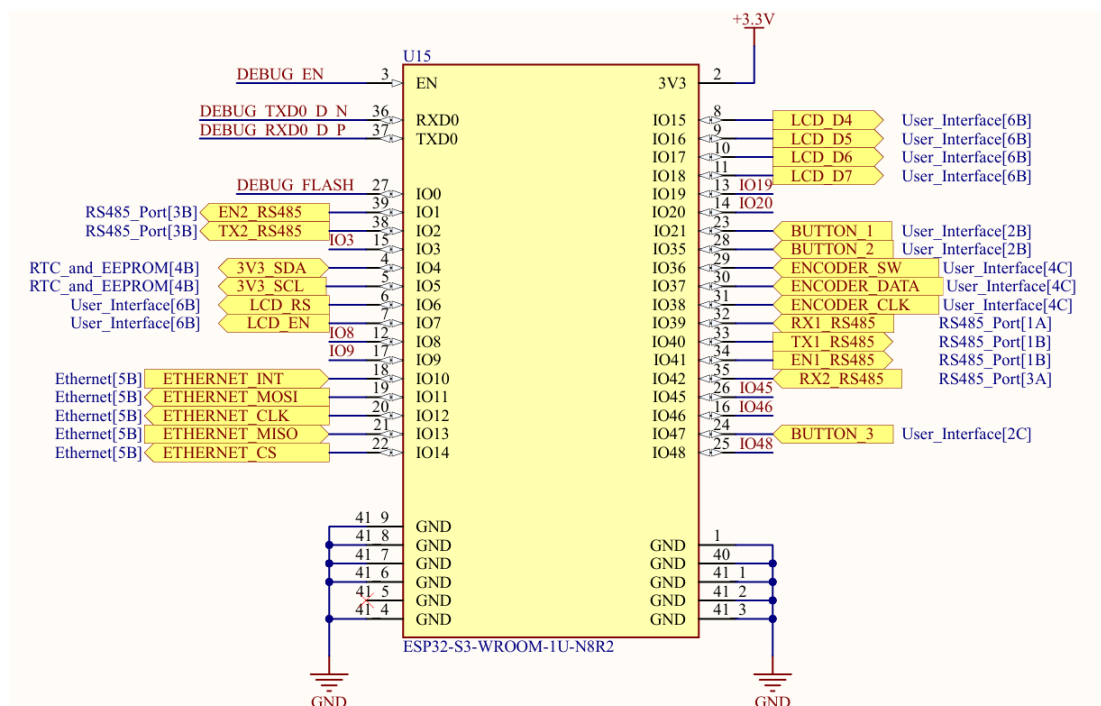
Nguồn DC 12-24V được đưa vào module hạ áp (buck converter) sử dụng IC TPS5430 để tạo ra điện áp 5V. Từ nguồn 5V tiếp tục hạ áp xuống 3.3V thông qua IC XL1509 hoặc LDO để cấp cho các các khối khác và vi điều khiển, nguồn 3.3V được sử

dụng chủ yếu trong hệ thống nên ta chọn IC XL1509 cho dòng ngõ ra ở khoảng rộng hơn (1-2A) thay vì sử dụng LDO ngõ ra chỉ khoảng 1A.

- **Nguồn 5V:** cấp cho LCD, khối cách ly RS485.
- **Nguồn 3.3V:** cấp cho vi điều khiển ESP32, module Ethernet, mạch debug cho ESP32, module RTC, EEPROM và module SD Card.

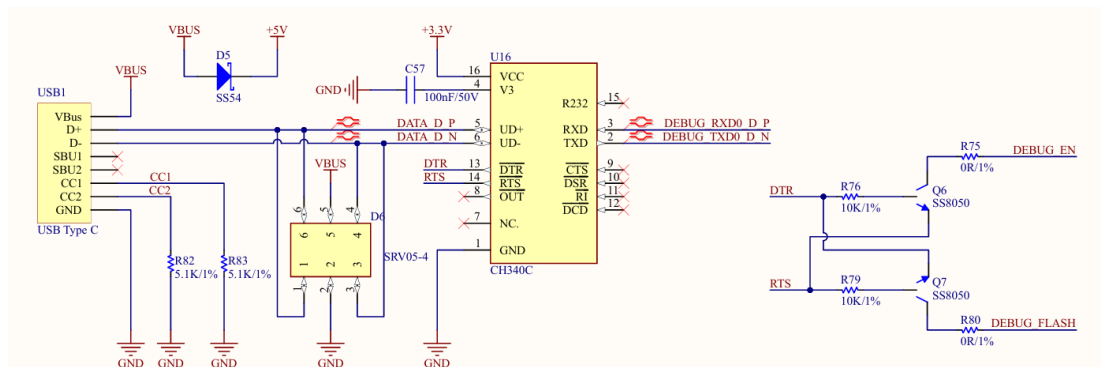
3.1.3 Chi tiết thiết kế phần cứng

3.1.3.1 Thiết kế khối vi điều khiển



Hình 3-4: khối vi điều khiển

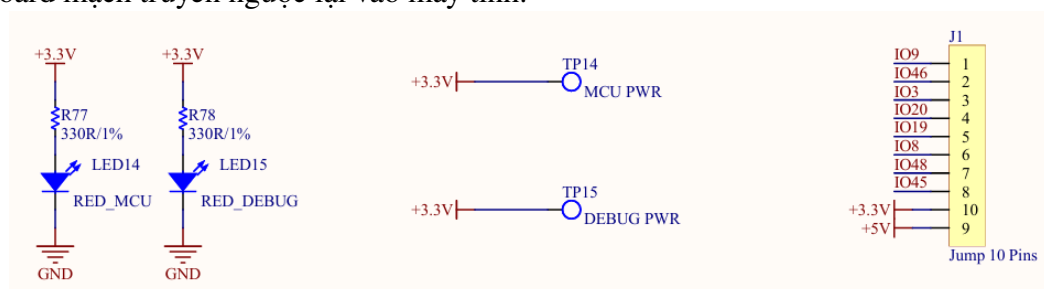
Sử dụng ESP32-S3 N8R2 làm bộ xử lý trung tâm cho các tác vụ tại thiết bị thu thập dữ liệu, với dung lượng bộ nhớ flash là 8MB và bộ nhớ RAM 2MB ngoại được tích hợp sẵn trong module, Vi điều khiển hỗ trợ các giao tiếp cơ bản như Wifi, Bluetooth/BLE giúp mở rộng khả năng giao tiếp trong quá trình phát triển thiết bị.



Hình 3-5: khối nạp code và Debug

Sử dụng IC CH340C làm IC debug chính cho thiết bị kết hợp với mạch nạp code tự động.

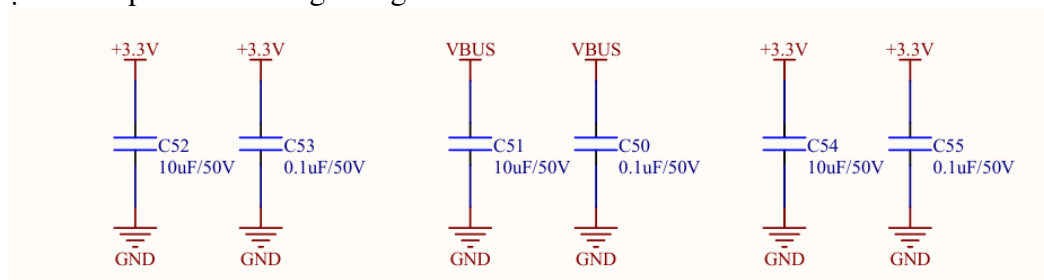
Cổng nạp sử dụng cổng type-C phổ biến và Diode giúp không cho điện áp cao từ board mạch truyền ngược lại vào máy tính.



Hình 3-6: khối kiểm tra nguồn cho vi điều khiển

Thiết kế hai đèn LED báo nguồn để người dùng có thể biết vi điều khiển đã có nguồn hay chưa.

Các chân chưa được sử dụng trong thiết kế mạch được đưa ra chân header giúp cho việc có thể phát triển trong tương lai.

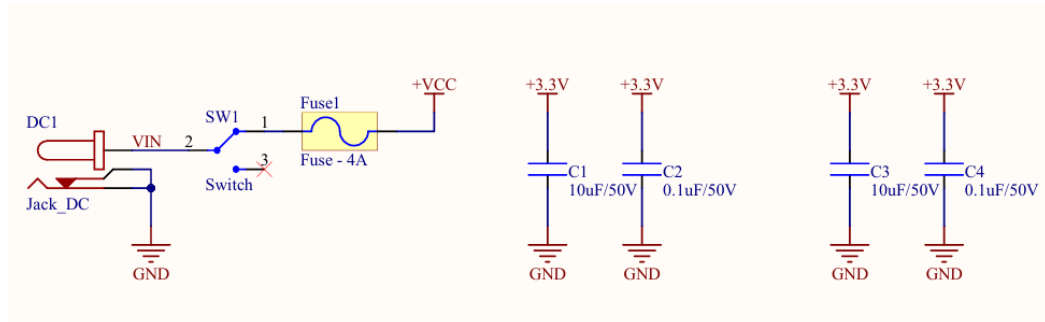


Hình 3-7: khối lọc nguồn đầu vào cho vi điều khiển

Tích hợp thêm các tu lọc nguồn đầu vào cho vi điều khiển, nhằm đảm bảo điện áp

cấp vào cho vi điều khiển được ổn định quanh mức cho phép.

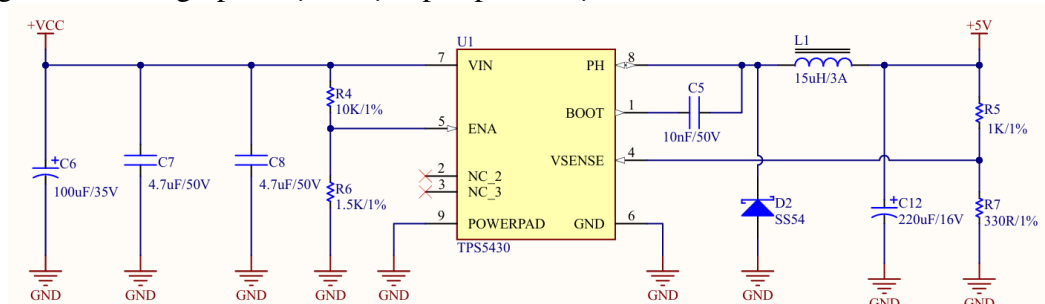
3.1.3.2 Thiết kế khối nguồn



Hình 3-8: khối nguồn đầu vào

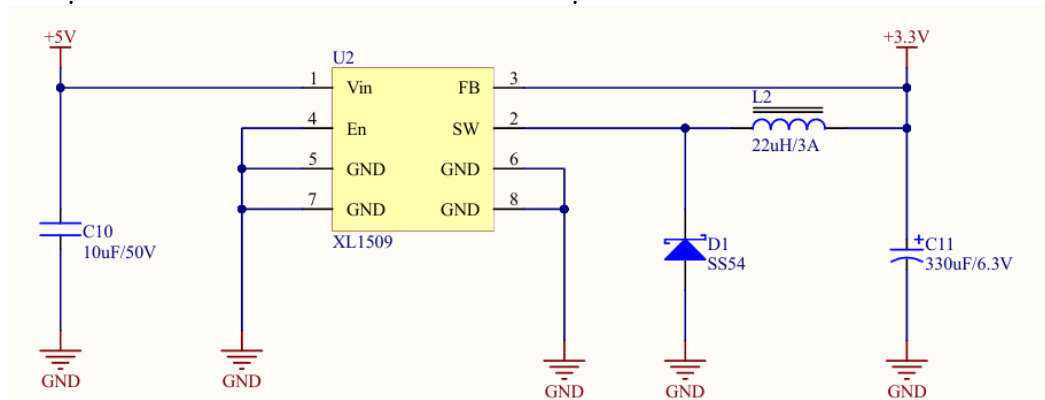
Sử dụng Adapter 12VDC/3A để cấp nguồn cho thiết bị thông qua Jack DC, dải điện áp đầu vào mà thiết bị có thể hoạt động là 12-24VDC.

Mạch có tích hợp sử dụng cầu chì giúp bảo vệ mạch khi quá dòng và các tụ lọc nguồn đầu vào giúp ổn định điện áp cấp vào mạch.



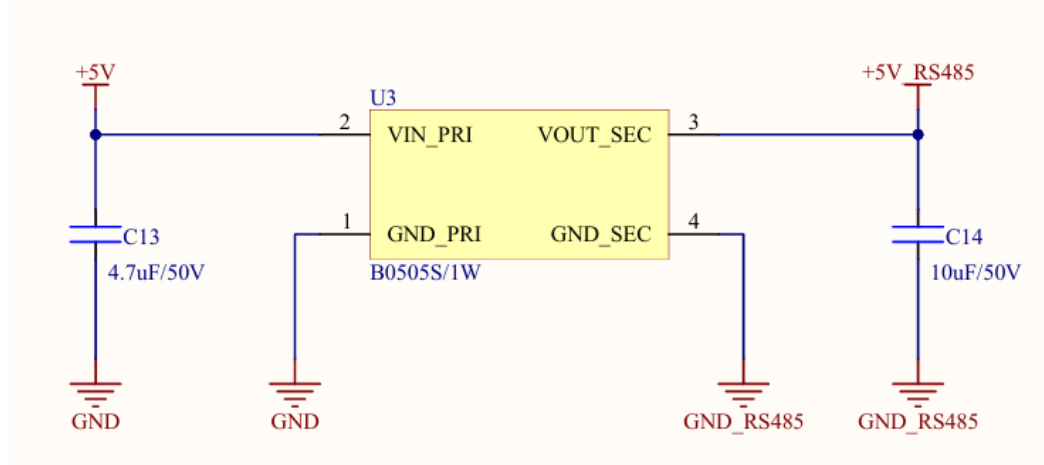
Hình 3-9: Khối nguồn hạ áp 5V

Sử dụng IC TPS5430 để hạ áp xuống 5V/3A ổn định tại ngõ ra với các thông số linh kiện tham khảo theo datasheet của linh kiện.



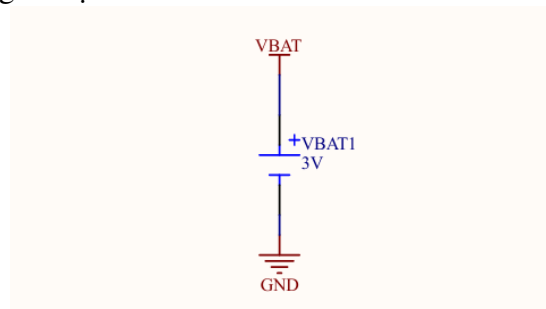
Hình 3-10: Khối nguồn hạ áp 3.3V

Sử dụng IC XL1509 để hạ áp xuống 3.3V/2A với các thông số linh kiện tham khảo theo datasheet của linh kiện, đây là nơi cung cấp nguồn chính cho các khối hoạt động trong mạch nên ta chọn IC cho dòng điện ngõ ra lớn hơn để có thể mở rộng sau này.



Hình 3-11: khối nguồn cách ly 5V

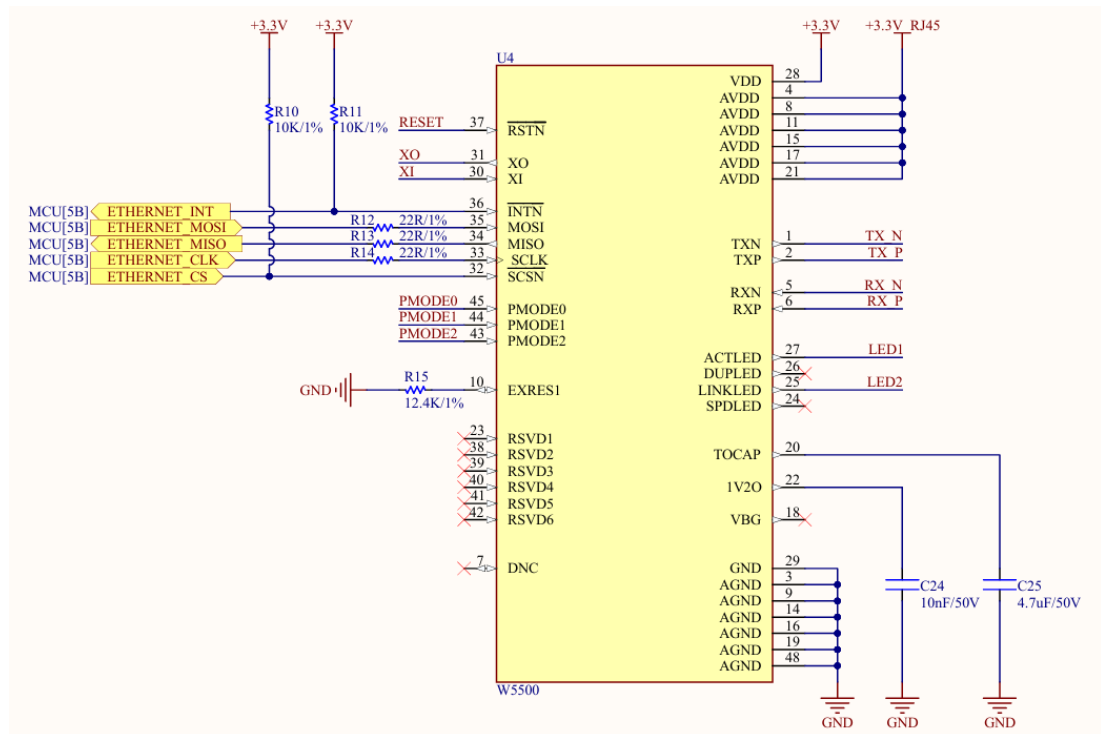
Sử dụng trực tiếp nguồn 5V có trên mạch để thực hiện tạo ra nguồn cách ly cho khối giao tiếp RS485, mục đích của việc cách ly là giúp cho các thành phần linh kiện trên mạch không bị tác động bởi các thiết bị bên ngoài kết nối với mạch thông qua khối RS485, tránh làm hỏng các linh kiện có trên mạch khi có các gai điện áp cao đi thẳng vào mạch, các tụ lọc nguồn giúp ổn định điện áp đầu vào cho các thành phần chính trong mạch hoạt động ổn định.



Hình 3-12: khối nguồn Pin rừ cho mạch RTC

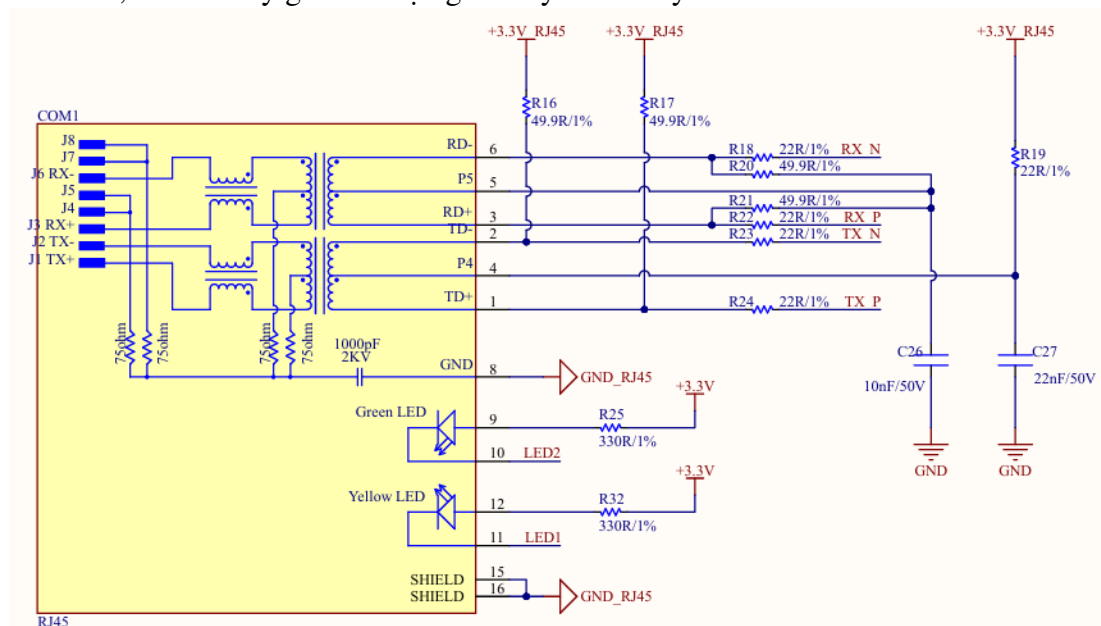
Sử dụng nguồn từ pin CR2032 cung cấp cho khối RTC, khối RTC sẽ hoạt động với nguồn cấp từ mạch nếu có nguồn adapter cấp cho mạch, khi adapter không cấp nguồn cho mạch thì khối RTC sẽ vẫn hoạt động dựa trên pin CR2032 này nhằm đảm bảo khối RTC vẫn hoạt động bình thường, giúp cung cấp thời gian chính xác tới khi nguồn điện chính từ adapter được cấp lại.

3.1.3.3 Thiết kế khối Ethernet



Hình 3-13: khối vi điều khiển cho Ethernet

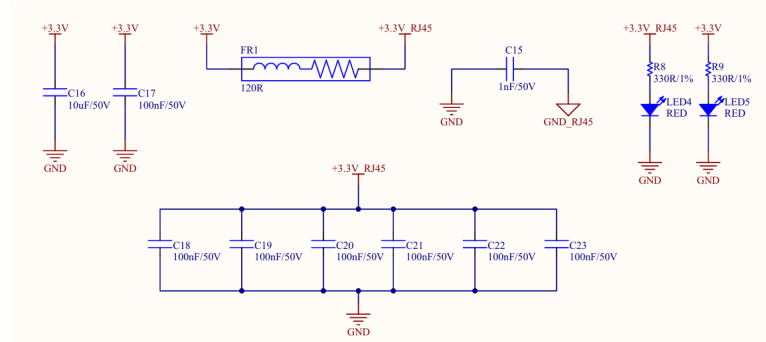
Sử dụng chip W5500 làm chip chính cho khối Ethernet, sử dụng W5500 ở chế độ Mac raw, tức là xử lý gói tin nhận/gửi ở layer 1 và layer 2.



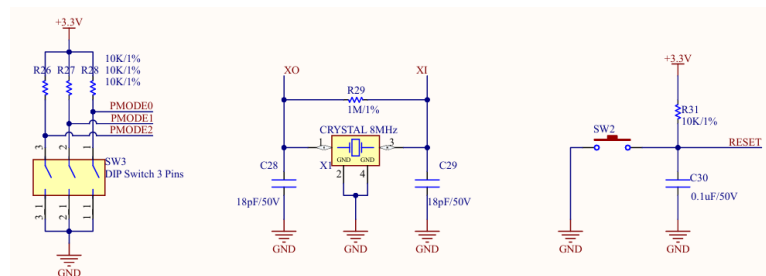
Hình 3-14: khối cổng kết nối cho Ethernet

Cổng RJ45 được chọn để truyền/nhận gói tin cho giao tiếp Ethernet, sử dụng các

điện trở kéo lên để giúp đường truyền tránh tình trạng floating, các điện trở hạn dòng và



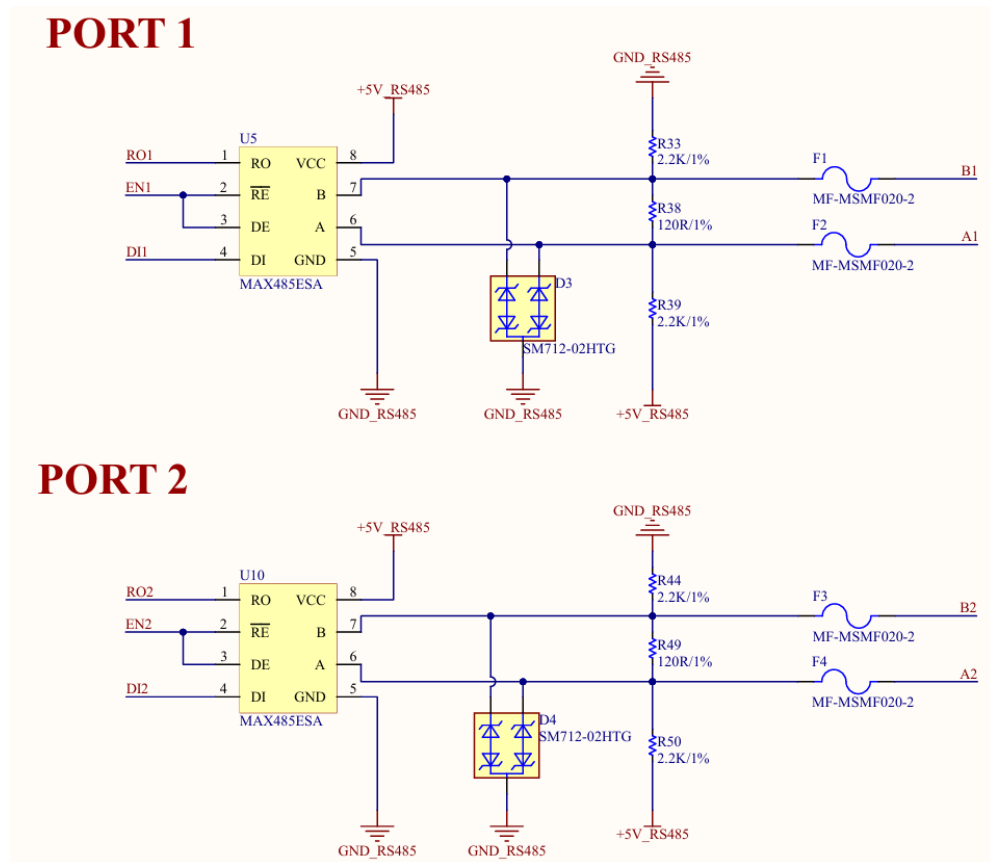
Hình 3-15: khối lọc nguồn đầu vào cho Ethernet



Hình 3-16: khối thạch anh và DIP Switch cho Ethernet

Dip switch đóng vai trò trong việc chọn mode hoạt động cho chip Ethernet. Thạch anh cung cấp xung clock 25MHz cho chip hoạt động. Tích hợp nút reset khi chip bị treo hoặc có lỗi.

3.1.3.4 Thiết kế khối giao tiếp RS485

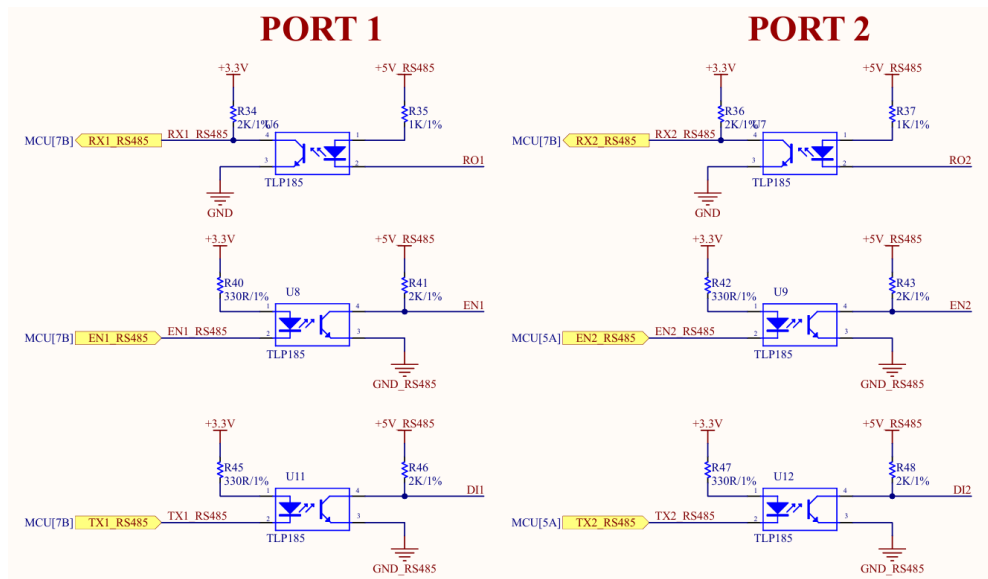


Hình 3-17: khối xử lý tín hiệu RS485

IC MAX485 đóng vai trò chính trong việc giao tiếp RS485 giữa vi điều khiển với các thiết bị modbus nói riêng và các thiết bị sử dụng chuẩn RS485.

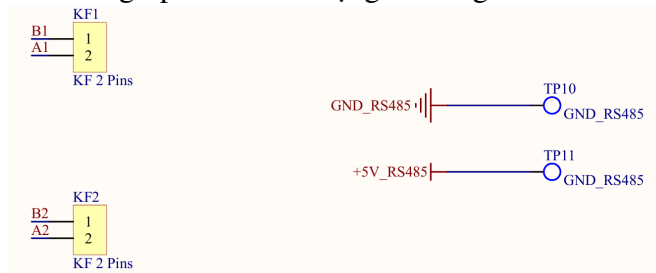
Hai đường tín hiệu A và B có sử dụng điện trở kéo lên và kéo xuống giúp tránh tình trạng floating khi đường truyền rảnh.

Tích hợp hai cầu chì tự phục hồi với dòng điện định mức là 0.75A và TVS, giúp bảo vệ mạch bên trong trước các gai điện áp cao từ ngoài đưa vào mạch.

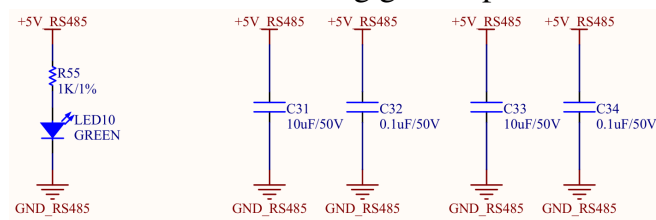


Hình 3-18: khối cách ly RS485

Mạch sử dụng Opto quang giúp cách ly mạch chính với khối giao tiếp RS485, sử dụng các điện trở kéo lên giúp tránh tình trạng floating.



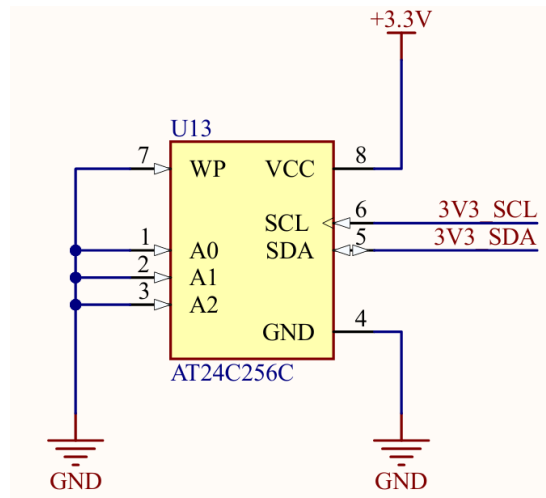
Hình 3-19: khối cổng giao tiếp RS485



Hình 3-20: khối lọc nguồn đầu vào và LED báo nguồn

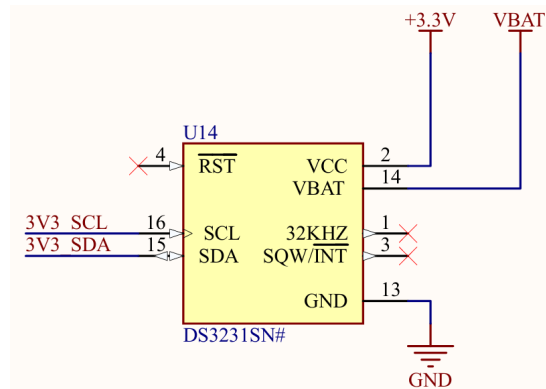
Các tụ điện lọc nguồn đầu vào giúp điện áp cấp cho các thành phần trong mạch hoạt động ổn định.

3.1.3.5 Thiết kế khối EEPROM và RTC



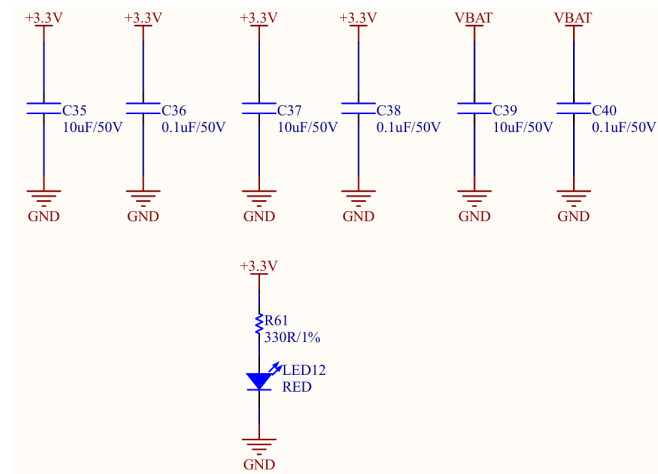
Hình 3-21: khối bộ nhớ EEPROM

Mạch có tích hợp sử dụng bộ nhớ EEPROM 256Kbit, giúp lưu trữ các thông tin liên quan tới cấu hình wifi cho vi điều khiển, sử dụng giao tiếp I2C với tốc độ 100KHz, giúp giảm tải lên bộ nhớ Flash của vi điều khiển.



Hình 3-22: khối RTC

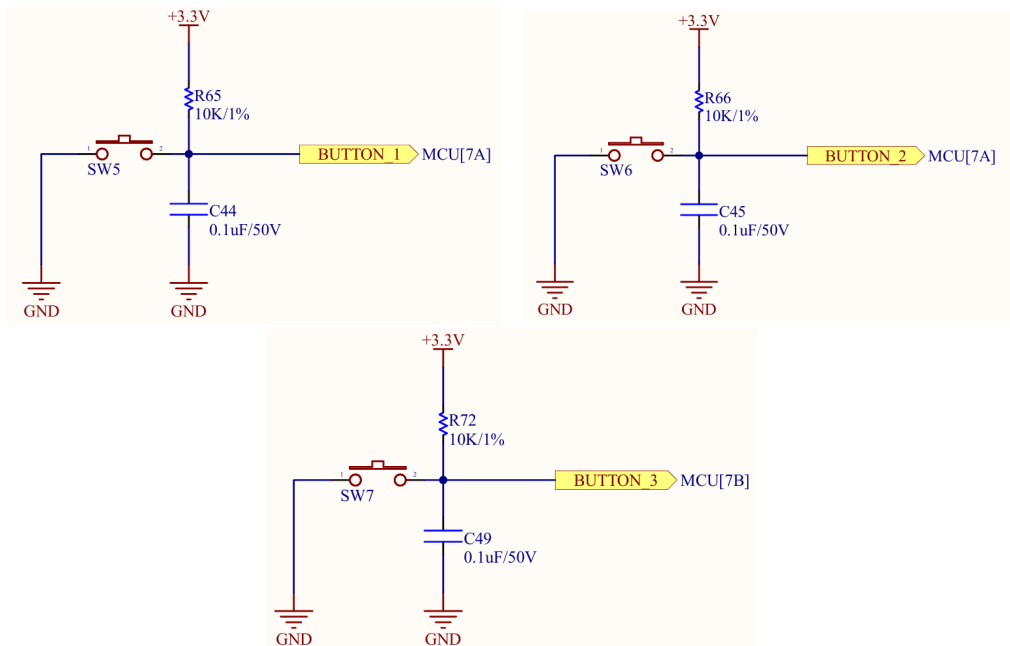
Khối RTC giúp lưu trữ thông tin thời gian, khi vi điều khiển kết nối được với Internet thì sẽ lấy thời gian từ NTP sau đó lưu vào trong RTC và sử dụng Pin CR2302 tích hợp, giúp IC vẫn có thể hoạt động ngay cả khi nguồn chính bị mất, giao tiếp I2C 100KHz sử dụng chung bus với bộ nhớ EEPROM. Khi vi điều khiển không thể kết nối được với internet thì sẽ sử dụng thời gian lưu sẵn trong khối RTC.



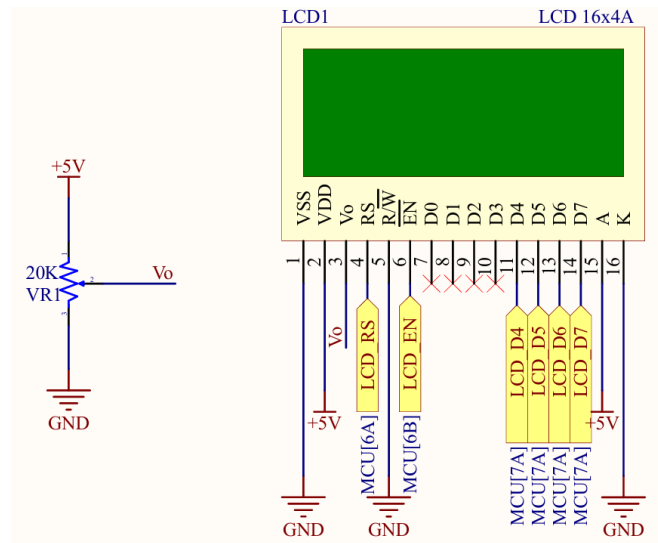
Hình 3-23: khối lọc nguồn đầu vào cho EEPROM và RTC

Thực hiện thiết kế khối lọc nguồn đầu vào dùng chung cho cả nguồn nuôi EEPROM và RTC .

3.1.3.6 Thiết kế khối giao tiếp với người dùng



Hình 3-24: khối nút nhấn tương tác với người dùng



Hình 3-25: màn hình LCD hiển thị

Mạch tích hợp ba nút nhấn lập trình cho phép người dùng có thể tương tác với thiết bị.

Màn hình LCD 16x04 hiển thị các thông tin cấu hình hiện có trên thiết bị, các tính năng của nút nhấn giúp tương tác và hiển thị thông tin trên LCD, giúp người dùng có thể điều chỉnh thông số và có thể cấu hình thông số tại chỗ mà không cần phải kết nối với máy tính.

Biến trở 20K Ohm giúp điều chỉnh hiển thị độ tương phản trên màn hình LCD.

CHƯƠNG 4 THIẾT KẾ VÀ THỰC HIỆN PHẦN MỀM

4.1 Thiết kế phần mềm cho thiết bị thu thập dữ liệu

4.1.1 Yêu cầu đặt ra cho phần mềm

Thiết bị được thiết kế cho mục đích thu thập dữ liệu theo thời gian thực, yêu cầu có thể xử lý nhiều tác vụ song song với nhau, cụ thể hơn các yêu cầu thiết kế bao gồm:

- **Tính ổn định khi hoạt động:** Yêu cầu quá trình thu thập dữ liệu từ các đồng hồ điện theo chu kỳ, dữ liệu định kỳ được ghi vào bộ nhớ tại thiết bị và định kỳ gửi dữ liệu lên Cloud, có thể chuyển đổi qua phương thức kết nối mạng có dây khi kết nối mạng không dây không khả dụng.
- **Khả năng mở rộng và phát triển:** Cấu trúc chương trình cần hỗ trợ dễ dàng tích hợp thêm tính năng mới hoặc thay đổi phần cứng mà không ảnh hưởng đến các chức năng hiện có trên thiết bị.
- **Dễ dàng cấu hình và cập nhật tính năng mới:** Cho phép nạp chương trình mới qua giao tiếp USB, trong quá trình phát triển thiết bị có thể in log ra console giúp thuận tiện trong việc phát hiện các lỗi tiềm ẩn, giúp việc nâng cấp và bảo trì hệ thống trở nên thuận tiện.
- **Thiết kế giao diện người dùng:** Cho phép người dùng có thể tương tác với thiết bị với mục đích cấu hình một số các thông số cho thiết bị hoạt động.
- **Có khả năng lưu trữ dữ liệu:** Thiết bị sau khi thu thập dữ liệu sẽ được lưu trữ ngay trên thiết bị, phòng trường hợp thiết bị mất kết nối mạng vẫn có thể lưu trữ được dữ liệu theo thời gian.

4.1.2 Phương pháp thực hiện chương trình

Để đáp ứng được yêu cầu quan trọng nhất là đáp ứng thời gian thực khi thu thập dữ liệu từ các đồng hồ điện, ngôn ngữ sử dụng chính trong quá trình viết phần mềm là C, với khả năng ngôn ngữ C có thể trực tiếp tác động lên phần cứng, kết hợp với sử dụng hệ điều hành FreeRTOS, chính điều này giúp đáp ứng thời gian thực hơn các ngôn ngữ khác. Phương pháp triển khai chương trình cụ thể cho từng yêu cầu ở trên như sau:

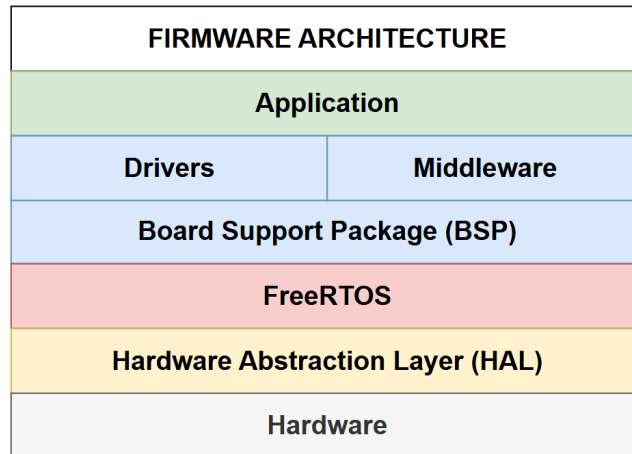
- **Tính ổn định khi hoạt động:** Sử dụng FreeRTOS để chia hệ thống thành các Task độc lập (Task thu thập dữ liệu, Task ghi Log, Task truyền tin), điều này giúp việc thu thập dữ liệu từ đồng hồ điện diễn ra đúng chu kỳ mà không bị ảnh hưởng

bởi việc truyền dữ liệu lên Cloud chậm. Phân tích logic kiểm tra trạng thái mạng. Hệ thống sẽ ưu tiên Wi-Fi, nếu mất kết nối Wi-Fi sẽ tự động Chuyển qua gửi dữ liệu thông qua Ethernet để đảm bảo dữ liệu luôn được đẩy lên Cloud.

- **Khả năng mở rộng và phát triển:** Mỗi tính năng mới được xây dựng như một thư viện (Middleware) riêng biệt, chỉ cần tích hợp vào hệ thống và tạo task dựa trên thư viện vừa tạo mà không can thiệp vào các Task cũ.
- **Dễ dàng cấu hình và cập nhật tính năng mới:** Sử dụng giao tiếp USB-to-UART, sử dụng thư viện in Log của ESP32 chuyển đổi để nạp chương trình và xuất Log ra terminal qua chuẩn UART của ESP32.
- **Thiết kế giao diện người dùng:** Thiết kế ra các giao diện cố định hoặc giao diện động dựa trên chức năng đang thực hiện. Màn hình sẽ hiển thị các mục được định nghĩa trước để người dùng có thể lựa chọn thông qua các nút nhấn và encoder tích hợp sẵn trên thiết bị giao tiếp bằng GPIO. Về phần cấu hình Wi-Fi cho thiết bị có thể được thực hiện thông một Web Server đơn giản, người dùng sẽ nhập SSID và Password bằng cách kết nối với Wi-Fi do ESP32 phát, giao diện nhập liệu Wi-Fi được viết bằng ngôn ngữ HTML đơn giản và được lưu vào bộ nhớ Flash của ESP32.
- **Có khả năng lưu trữ dữ liệu:** Sử dụng thẻ nhớ SD Card được tích hợp thêm vào thiết bị và sử dụng giao thức SPI để ghi dữ liệu vào SD Card, chức năng ghi dữ liệu vào SD Card được tách thành một task riêng biệt chạy song song với các task khác trong chương trình, điều này giúp cho khi thiết bị không có kết nối mạng, không thể gửi dữ liệu lên Cloud, thì dữ liệu vẫn được ghi vào thẻ nhớ. Người dùng có thể đọc được file ghi dữ liệu này dưới dạng file Excel, ứng với mỗi dữ liệu sẽ kèm theo mốc thời gian tại thời điểm ghi dữ liệu đó.

4.1.3 Lưu đồ giải thuật chương trình

4.1.3.1 Kiến trúc tổng quát về tổ chức phần mềm



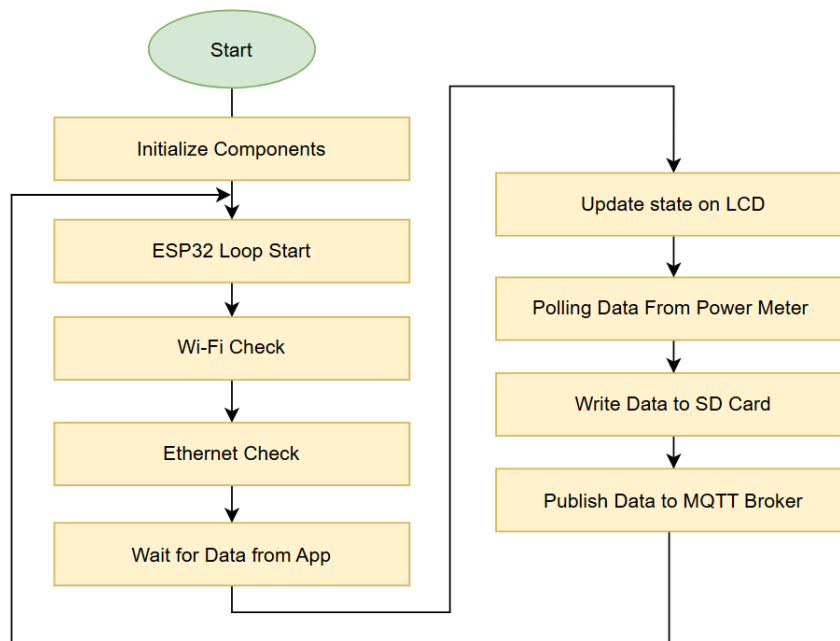
Hình 4-1: Tổng quát kiến trúc phần mềm của thiết bị

- **Application:** Lớp cao nhất, chứa logic thực hiện cốt lõi của hệ thống, nơi thực hiện các hoạt động theo yêu cầu của người dùng như thực hiện chu kỳ thu thập dữ liệu từ đồng hồ điện, Xử lý logic ghi dữ liệu vào bộ nhớ cục bộ khi mất mạng và đồng bộ hóa dữ liệu với Cloud khi có kết nối mạng, quản lý giao diện cấu hình thông số hệ thống và hiển thị trạng thái hoạt động của thiết bị, nơi định nghĩa các trang có thể hiển thị trên màn hình LCD.
- **Drivers và Middleware:** Drivers điều khiển các thiết bị ngoại vi cụ thể thông qua thư viện phần mềm, thực hiện viết phần mềm cấu hình, mô tả cách thức hoạt động cho các khối chức năng trên thiết bị. Middleware cung cấp các thư viện giao thức và dịch vụ thực hiện. Bao gồm các bộ thư viện như Modbus Stack, stack TCP/IP trong LwIP của ESP32 cho Ethernet/Wi-Fi hoặc các thư viện mã hóa bảo mật khi gửi dữ liệu lên Cloud.
- **Board Support Package (BSP):** Tập hợp các định nghĩa phần cứng cụ thể cho thiết bị thông qua phần mềm, cấu hình các chân kết nối và thiết lập môi trường hoạt động cho board mạch thực tế đang thiết kế, giúp kết nối giữa phần cứng và phần mềm.
- **FreeRTOS:** Đây là lớp nền tảng, chịu trách nhiệm quản lý đa nhiệm, cấp phát bộ nhớ, đồng bộ hóa tiến trình và quản lý thời gian. FreeRTOS được tích hợp sẵn trong môi trường phát triển và được sử dụng xuyên suốt trong chương trình

chính giúp điều phối hoạt động giữa các task đã khởi tạo

- **Hardware Abstraction Layer (HAL):** Lớp HAL giúp tách biệt phần mềm với phần cứng cụ thể, cung cấp các giao diện trừu tượng để truy xuất phần cứng, được tích hợp sẵn trong bộ thư viện kèm theo khi sử dụng thư viện IDF của espressif. Cho phép phần mềm tương tác với phần cứng (như đọc/ghi GPIO, cấu hình bộ định thời) mà không cần can thiệp trực tiếp vào địa chỉ bộ nhớ của chip. Điều này giúp mã nguồn dễ dàng chuyển đổi sang các dòng chip khác trong tương lai nếu cần.
- **Hardware:** Bao gồm các tài nguyên phần cứng như vi điều khiển, bộ nhớ và các ngoại vi tích hợp, là cơ sở cho các tầng phần mềm phía trên hoạt động. Bao gồm vi điều khiển chính (ESP32-S3), các module tích hợp trên thiết bị (W5500 cho Ethernet), các ngoại vi lưu trữ (thẻ nhớ SD), và các IC hỗ trợ đọc dữ liệu từ đồng hồ điện kết nối qua RS485. Thực hiện các thao tác điện lý như đóng cắt tín hiệu, lưu trữ vật lý và truyền nhận dữ liệu thô qua các chân GPIO, UART, SPI.

4.1.3.2 Lưu đồ giải thuật tổng quát



Hình 4-2: Lưu đồ giải thuật thực hiện tổng quát của hệ thống

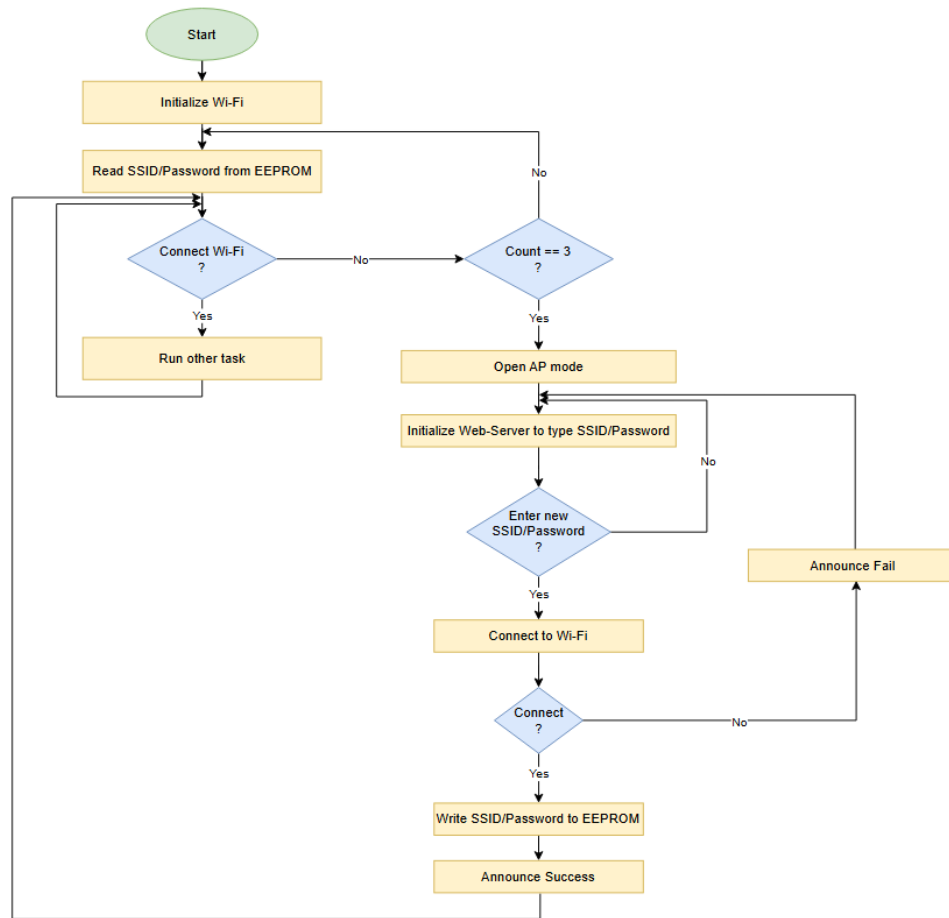
Lưu đồ này mô tả một cách tổng quát các hành vi có trong hệ thống, bao gồm:

- **Start:** khởi đầu của chương trình ngay khi thiết bị được cấp nguồn hoặc khởi

động lại (Reset).

- **Initialize Components:** Gọi các hàm khởi tạo của từng thành phần có trong thiết bị, gồm: Wi-Fi, Ethernet, Modbus RTU, Modbus TCP, SD Card, MQTT, User Interface
- **ESP32 Loop Start:** vòng lặp tuần hoàn, đảm bảo các tác vụ kiểm tra mạng, thu thập và gửi dữ liệu được thực hiện lặp đi lặp lại theo đúng trình tự.
- **Wi-Fi Check::** Luôn kiểm tra tình trạng kết nối Wi-Fi trong quá trình thiết bị hoạt động, có thể chuyển đổi qua Ethernet khi mất mạng.
- **Ethernet Check::** Luôn kiểm tra tình trạng kết nối Ethernet có dây trong quá trình thiết bị hoạt động, khi Wi-Fi có thể chuyển đổi qua sử dụng Ethernet.
- **Wait for Data from App:** Nhận gói tin chứa các thanh ghi cấu hình của các đồng hồ điện mới thêm vào hệ thống, hoặc xóa khỏi hệ thống. Các thanh ghi này sẽ được phục vụ cho việc thu thập dữ liệu của thiết bị.
- **Polling Data Power Meter:** Chức năng chính của thiết bị, thu thập dữ liệu định kỳ các đồng hồ điện. Sử dụng giao thức Modbus RTU để gửi lệnh truy vấn (Request) đến các đồng hồ điện và nhận về (Response) các thông số định lượng như điện áp, dòng điện, công suất và điện năng tiêu thụ.
- **Write Data to SD Card:** Lưu trữ dữ liệu cục bộ ngay tại thiết bị. Dữ liệu vừa thu thập được sẽ được ghi vào thẻ nhớ SD. Việc này đảm bảo tính an toàn dữ liệu, giúp lưu trữ thông tin theo thời gian thực ngay cả khi hệ thống mất kết nối mạng.
- **Publish Data to MQTT Broker:** Truyền tải thông tin đã xử lý lên máy chủ quản lý trung tâm. Đóng gói dữ liệu và gửi lên Cloud theo các giao thức truyền thông mạng.

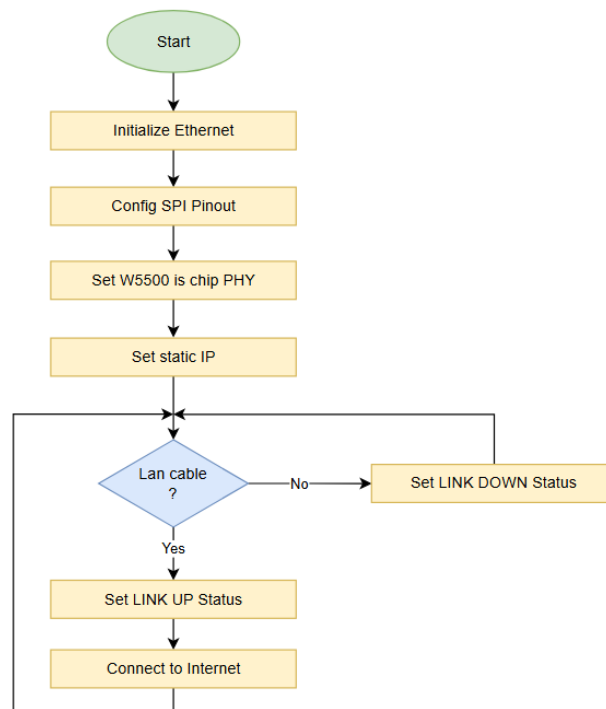
4.1.4 Lưu đồ giải thuật chi tiết



Hình 4-3: Lưu đồ giải thuật cấu hình Wi-Fi

- **Initialize Wi-Fi:** Khởi tạo trình các biến cấu hình cho Wi-Fi
- **Read SSID/Password from EEPROM:** Đọc thông tin mạng: tên Wi-Fi và mật khẩu đã lưu từ bộ nhớ EEPROM.
- **Connect Wi-Fi?:** Kiểm tra xem có kết nối thành công với thông tin đã đọc hay không.
Yes: Chuyển sang thực hiện các tác vụ chính khác
No: Kiểm tra số lần thử kết nối ($\text{Count} == 3$?). Nếu chưa đủ 3 lần, hệ thống sẽ quay lại thử lại.
- **Open AP mode:** Nếu thử lại 3 lần thất bại, thiết bị sẽ tự phát một mạng Wi-Fi riêng (Access Point) để người dùng kết nối vào.
- **Initialize Web-Server:** Khởi tạo một trang web nội bộ để người dùng có thể nhập tên Wi-Fi và mật khẩu mới.

- **Enter new SSID/Password ?**: Đợi người dùng nhập và gửi thông tin mạng mới từ trình duyệt.
- **Connect to Wi-Fi**: Thiết bị dùng thông tin mới vừa nhận được để thử kết nối lại.
Yes - Write SSID/Password to EEPROM: Lưu thông tin mạng mới vào bộ nhớ để sử dụng cho những lần khởi động sau.
No - Announce Success: Thông báo kết nối thành công và quay lại quy trình vận hành bình thường.



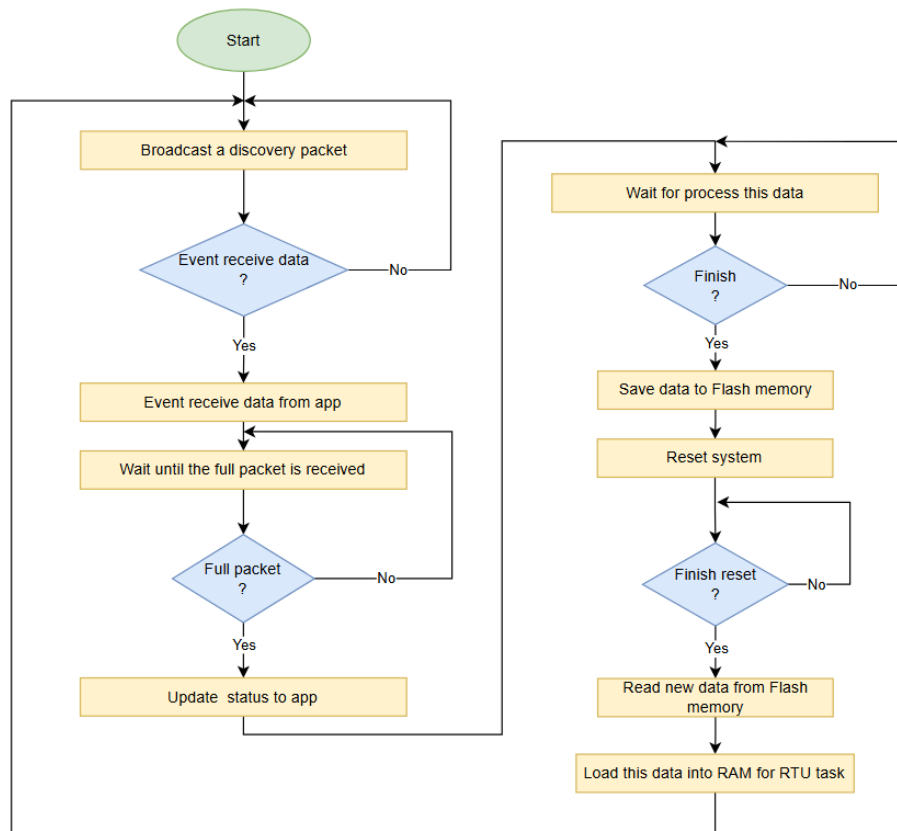
Hình 4-4: Lưu đồ giải thuật cấu hình Ethernet

- **Initialize Ethernet**: Bắt đầu quá trình khởi tạo các biến liên quan cho kết nối Ethernet.
- **Config SPI Pinout**: Cấu hình các chân tín hiệu của giao thức SPI: MOSI, MISO, SCK, CS để ESP32 có thể giao tiếp với chip W5500.
- **Set W5500 is chip PHY**: Chỉ định chip W5500 đóng vai trò là bộ điều khiển lớp vật lý PHY để xử lý tín hiệu mạng dây, sử dụng W5500 với chế độ MACRAW trong ESP32.
- **Set static IP**: Thiết lập địa chỉ IP tĩnh cố định cho thiết bị.
- **Lan cable ?**: Kiểm tra xem dây mạng LAN đã được cắm vào cổng RJ45 hay chưa.

No - Set LINK DOWN Status: Nếu chưa có cáp, hệ thống xác lập trạng thái mất kết nối và tiếp tục vòng lặp kiểm tra.

Yes - Set LINK UP Status: Nếu dây mạng đã cắm, thì xác nhận trạng thái kết nối vật lý thành công.

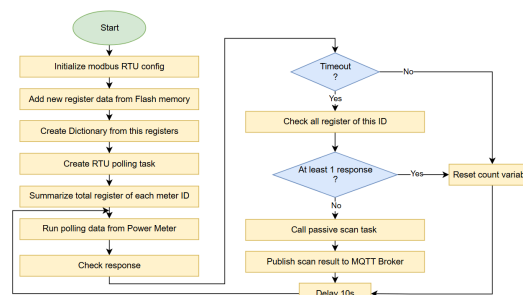
- **Connect to Internet:** Thiết bị thực hiện các giao thức mạng để kết nối ra Internet thông qua đường truyền Ethernet đã thiết lập.



Hình 4-5: Lưu đồ giải thuật đợi nhận data từ App qua BLE

- **Broadcast a discovery packet:** Thiết bị phát các gói tin quảng bá ra không gian mạng để thông báo sự hiện diện của mình.
- **Event receive data ?:** Hệ thống kiểm tra xem có bất kỳ phản hồi hoặc dữ liệu nào gửi tới sau khi phát quảng bá không.
No: Quay lại tiếp tục chờ kết nối từ App.
Yes: Ghi nhận sự kiện nhận dữ liệu từ App.
- **Wait until the full packet is received:** chờ để đảm bảo nhận đầy đủ tất cả các phần của gói tin từ App gửi tới.

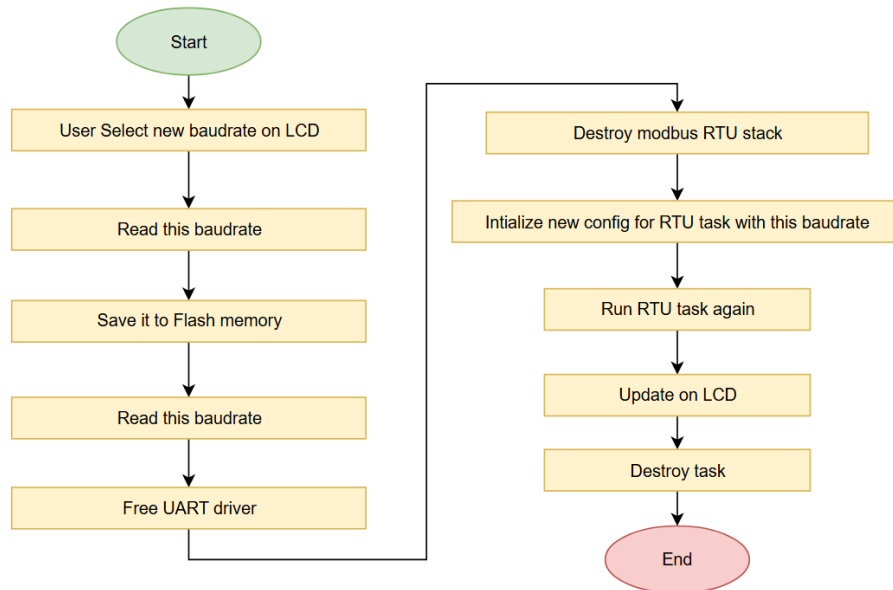
- **Full packet ?**: Kiểm tra xem toàn bộ nội dung gói tin đã được nhận đủ chưa, dựa vào kiểm tra dấu ngoặc vuông cuối cùng của gói tin JSON.
- No**: Tiếp tục nhận và chờ.
- Yes**: Gửi phản hồi thành công về App và bắt đầu xử lý lưu vào bộ nhớ Flash.
- **Wait for process this data**: Hệ thống tiến hành xử lý dữ liệu.
- **Finish?**: Xác nhận việc xử lý dữ liệu đã hoàn tất.
- **Save data to Flash memory**: Lưu các thông số mới vào bộ nhớ Flash.
- **Reset system**: Khởi động lại hệ thống.
- **Finish reset ?**: Đợi hệ thống hoàn tất quá trình tự kiểm tra sau khi khởi động lại.
- **Read new data from Flash memory**: Truy xuất các thông số vừa được lưu trước đó từ Flash.
- **Load this data into RAM for RTU task**: Nạp dữ liệu vào RAM để tác vụ Modbus RTU bắt đầu hoạt động với tập thanh ghi mới nhất.



Hình 4-6: Lưu đồ giải thuật thu thập dữ liệu từ đồng hồ điện

- **Initialize modbus RTU config**: Thiết lập các thông số cấu hình cơ bản cho giao thức Modbus RTU (như cổng UART, parity, stop bits).
- **Add new register data from Flash memory**: Truy xuất danh sách tất cả các thanh ghi được lưu trữ sẵn trong bộ nhớ Flash đưa vào RAM.
- **Create Dictionary from this registers**: Tạo danh sách thanh ghi thành dạng "Dictionary" theo chuẩn chuẩn của thư viện esp-modbus dựa trên dữ liệu vừa được đưa vào RAM.
- **Create RTU polling task**: Tạo một task riêng cho việc thu thập dữ liệu từ các đồng hồ điện.
- **Run polling data from Power Meter**: Thực hiện việc gửi lệnh đọc và nhận dữ liệu thực tế từ đồng hồ đo điện thông qua đường truyền Modbus.

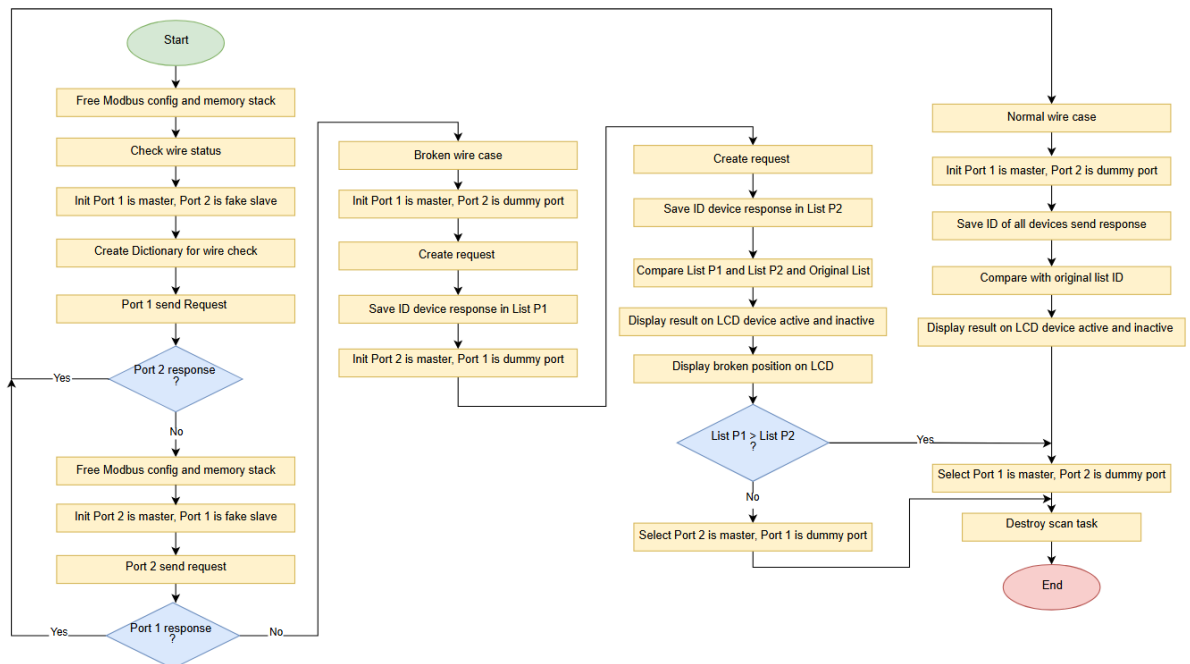
- **Delay 10s:** Cứ 10 giây thì chạy task thu thập dữ liệu đồng hồ điện một lần.



Hình 4-7: Lưu đồ giải thuật thực hiện thay đổi tốc độ baudrate

- **User Select new baudrate on LCD:** Người dùng chọn một giá trị tốc độ baud mới thông qua giao diện màn hình LCD.
- **Read this baudrate:** Hệ thống ghi nhận và xử lý giá trị đã chọn từ đầu vào.
- **Save it to Flash memory:** Lưu tốc độ baud mới vào bộ nhớ Flash để đảm bảo thông tin không bị mất khi mất điện hoặc khởi động lại.
- **Read this baudrate:** Hệ thống đọc lại giá trị vừa lưu từ Flash để kiểm tra và xác nhận.
- **Free UART driver:** Giải phóng driver UART hiện tại để nạp cấu hình mới.
- **Destroy modbus RTU stack:** Hủy bỏ ngăn xếp (stack) Modbus RTU cũ để chuẩn bị cho một thiết lập mới.
- **Initialize new config for RTU task with this baudrate:** Khởi tạo cấu hình mới cho tác vụ RTU với tốc độ baud vừa được cập nhật.
- **Run RTU task again:** Khởi chạy lại tác vụ truyền thông Modbus RTU với các thông số cấu hình mới.
- **Update on LCD:** Cập nhật trạng thái hoặc tốc độ baud mới lên màn hình LCD cho người dùng.
- **Destroy task:** Giải phóng task cấu hình tạm thời sau khi hệ thống đã cập nhật

thành công.



Hình 4-8: Lưu đồ giải thuật chức năng scan đường dây

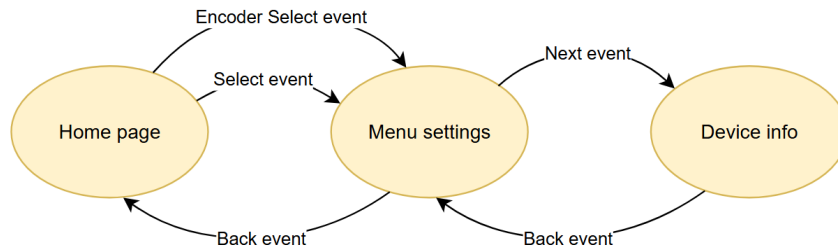
Ý tưởng để thực hiện task scan là thực hiện thay phiên nhau hai port gửi các gói tin ra đường truyền và tách lấy các ID từ các gói tin phản hồi gửi về, ta chia quá trình scan này thành hai giai đoạn, giai đoạn một sẽ thực hiện gửi các request nhằm xác định đường truyền có đang bị gián đoạn không, giai đoạn hai ta sẽ thực hiện việc tìm chính xác tại điểm gián đoạn đó ở đoạn nào trên đường truyền, cụ thể như sau:

Giai đoạn 1: thực hiện kiểm tra đường truyền có đứt dây tại đoạn nào không:

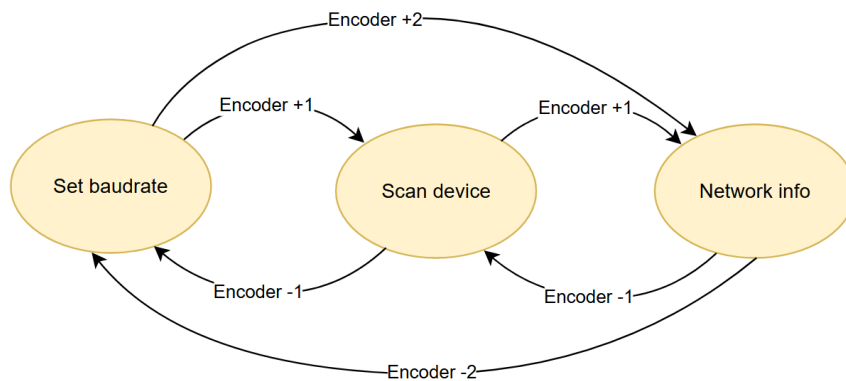
- Ban đầu sẽ thực xóa các task liên quan tới modbus hiện tại.
- cấu hình Port 1 làm Master Port 2 làm Slave, Port 1 tạo các request và đợi Port 2 Slave phản hồi
- Nếu Slave 2 phản hồi thì ta kết luận đường truyền không đứt.
- Nếu Slave 2 không phản hồi tức là Timeout, ta chuyển sang Port 2 làm Master và Port 1 làm Slave.
- Tương tự với trường hợp làm với Port 1, ta cũng chờ Port 1 phản hồi.
- Nếu Slave 1 phản hồi thì ta kết luận đường truyền không đứt.
- Nếu Slave 1 không phản hồi ta kết luận đường truyền đang gián đoạn.

Giai đoạn 2: thực hiện kiểm tra đường truyền gián đoạn tại đầu (nếu có):

- cấu hình Port 1 làm Master Port 2 làm Slave, Port 1 tạo các request và đợi các đồng hồ phản hồi.
- Ta tiến hành tách lấy phần ID từ những gói tin phản hồi và lưu vào List P1 (Tức là List gồm các ID phản hồi về Port 1)
- cấu hình Port 2 làm Master Port 1 làm Slave, Port 2 tạo các request và đợi các đồng hồ phản hồi.
- Ta tiến hành tách lấy phần ID từ những gói tin phản hồi và lưu vào List P2 (Tức là List gồm các ID phản hồi về Port 2)
- Khi này ta sẽ tiến hành so sánh ID có trong các List với List chứa các ID gốc đang có trong bảng tra của thiết bị.
- Nếu cả hai port đều không nhận được response từ một ID tức là tại đó thiết bị đang không hoạt động.
- Nếu port 1 chỉ tìm thấy các ID đầu của List chứa ID gốc và port 2 đang chứa các ID ở cuối List gốc, thì ta có thể kết luận vị trí gián đoạn đang ở giữa hai ID cuối của mỗi List.
- Nếu so sánh với List ID gốc mà port 1 không nhận được scan nào nhưng port thì có đủ tức là đường truyền đang bị gián đoạn tại đầu port 1 và ngược lại ta kết luận với port 2.
- Khi này sau khi đã có được kết quả scan đường truyền thì ta sẽ dựa trên kết quả này để chọn port cho việc thu thập dữ liệu
- Nếu đường truyền ổn định thì ta chọn mặc định port 1 làm port thu thập dữ liệu.
- Nếu đường truyền gián đoạn thì ta sẽ chọn port có số lượng ID thu thập được lớn hơn làm port thu thập dữ liệu.
- Sau cùng ta thực xóa stack của task scan và khôi phục lại task thu thập dữ liệu ban đầu với kết quả port mà ta vừa chọn sau khi scan đường truyền.



Hình 4-9: State machine mô tả chuyển trang của LCD



Hình 4-10: State machine mô tả trang Menu settings

Màn hình LCD hiển thị ba trang chính, gồm: Home, Menu settings và Device info. Các tín hiệu được sử dụng để điều hướng các trang đến từ các nút nhấn, vị trí quay của encoder và nút nhấn được tích hợp trên encoder thông qua các sự kiện:

- **Select event và Encoder Select event:** đứng tại trang Home và nhận được tín hiệu này sẽ vào trang Menu settings.
- **Next event:** Chuyển qua trang kế tiếp.
- **Back event:** Lùi về một trang trước trang đang đứng
- **Encoder+n:** vị trí xoay tối hiện tại của encoder so với vị trí ban đầu tăng lên n đơn vị.
- **Encoder-n:** vị trí xoay tối hiện tại của encoder so với vị trí ban đầu giảm xuống n đơn vị.

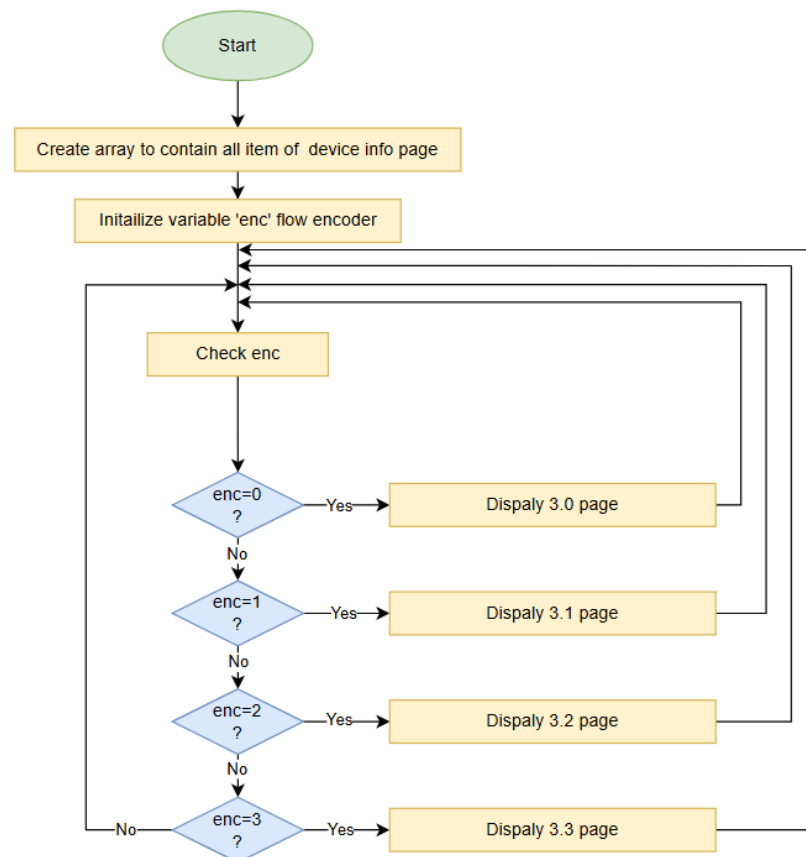
Mục Home thể hiện các thông tin trạng thái kết nối với Wi-Fi, Ethernet, Bluetooth và thời gian hiện tại.

Mục Menu settings được sử dụng để hiển thị cho các mục chức năng chính và di chuyển giữa các mục sẽ dựa trên vị trí hiện tại của encoder so với vị trí ban đầu trước

khi quay, mà con trỏ trên màn hình sẽ quyết định di chuyển lên hay di chuyển xuống theo ý người dùng, các chức năng gồm:

- **Set baudrate:** Cài đặt tốc độ baudrate cho thiết bị trong quá trình thu thập dữ liệu từ các đồng hồ điện.
- **Scan device:** Thực hiện chức năng scan đường truyền khi người dùng nhấn chọn vào mục này.
- **Network info:** Sử dụng để xem các thông tin về mạng kết nối của thiết bị.

Mục Device info được sử dụng để hiển thị các thông như ID để phục vụ cho việc kết nối và gửi dữ liệu lên Web người dùng và các thông tin như tên thiết bị, thời gian hoạt động của thiết bị, phiên bản hiện tại của phần mềm và phần cứng của thiết bị.



Hình 4-11: Lưu đồ giải thuật mô tả hoạt động của trang Device info

Trang này sử dụng với chức năng xem các thông tin của thiết bị.

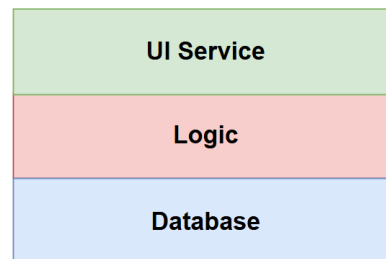
Ý tưởng sử dụng một biến dùng để đếm số nấc của encoder và di chuyển đến vị trí tiếp theo.

Khi lần encoder theo chiều kim đồng hồ thì biến đếm sẽ tăng lên và di chuyển xuống các mục phía dưới.

Ngược lại khi lần ngược chiều kim đồng hồ thì biến đếm sẽ giảm và trở lại các vị trí trang ban đầu.

4.2 Thiết kế phần mềm App nhập dữ liệu

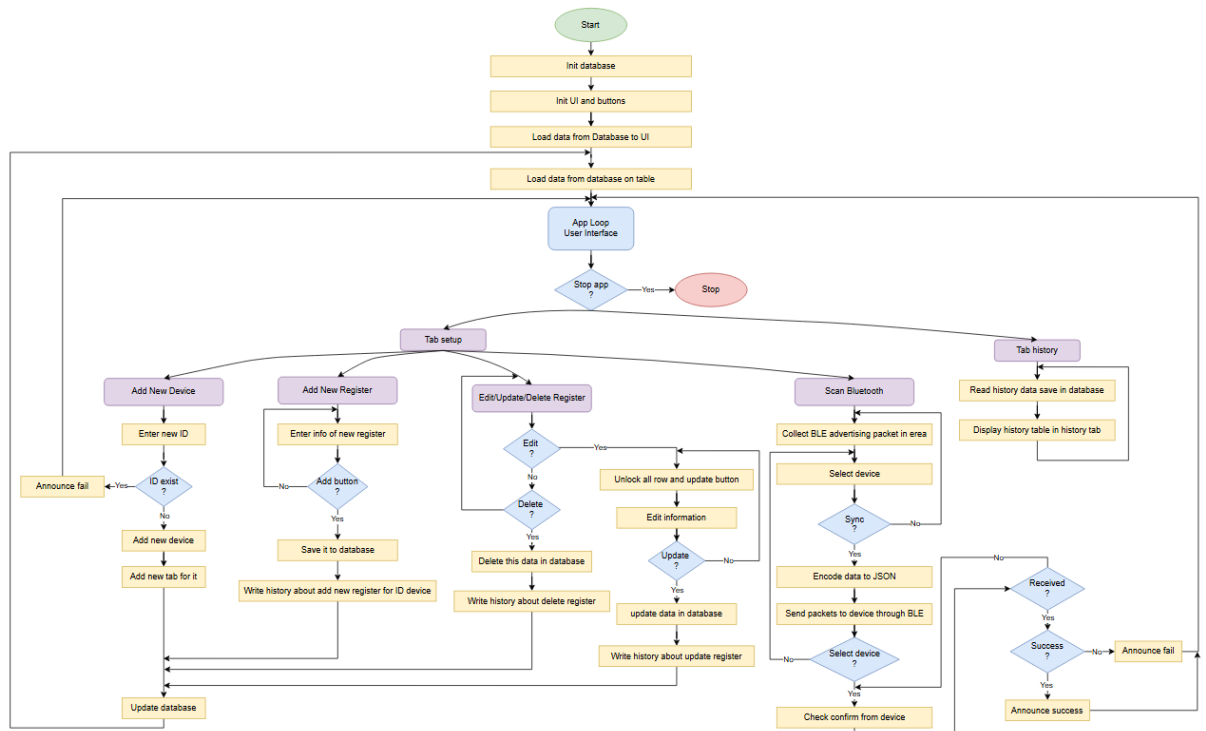
4.2.1 Thiết kế phần mềm



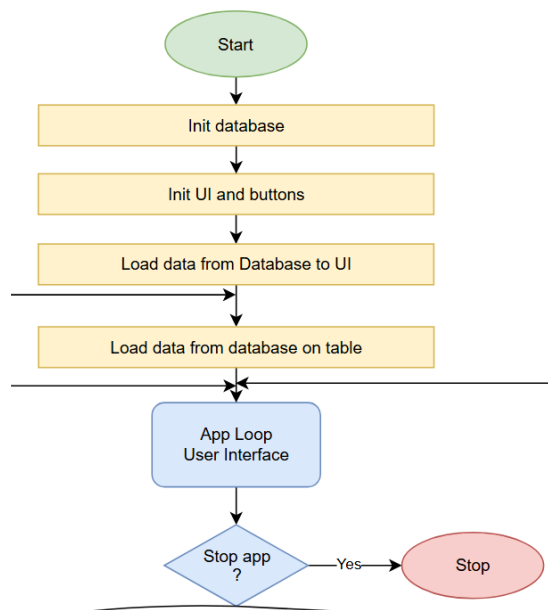
Hình 4-12: Sơ đồ khối tổng quát của phần mềm App

Cấu trúc của giải thuật App gồm có ba phần thực hiện các tác vụ hỗ trợ cho phần còn lại. Trong đồ án này sử dụng thư viện pyQT sử dụng ngôn ngữ Python, được phát triển lên từ thư viện C++ QT, thư viện này hỗ trợ cho phép người dùng tạo ra các ô nhập liệu, nút bấm và dựng cửa sổ giao diện bằng cách sử dụng các API đã được phát triển.

- **UI Service:** render giao diện người dùng bằng thư viện pyQT.
- **Logic:** Sử lý các tác vụ lưu thông tin người dùng nhập, xử lý dữ liệu, gửi dữ liệu xuống thiết bị.
- **Database:** Sử dụng database SQLite để lưu trữ các thông tin mà người dùng nhập và lịch sử nhập liệu và gửi dữ liệu.



Hình 4-13: Lưu đồ giải thuật chi tiết cho giao diện App

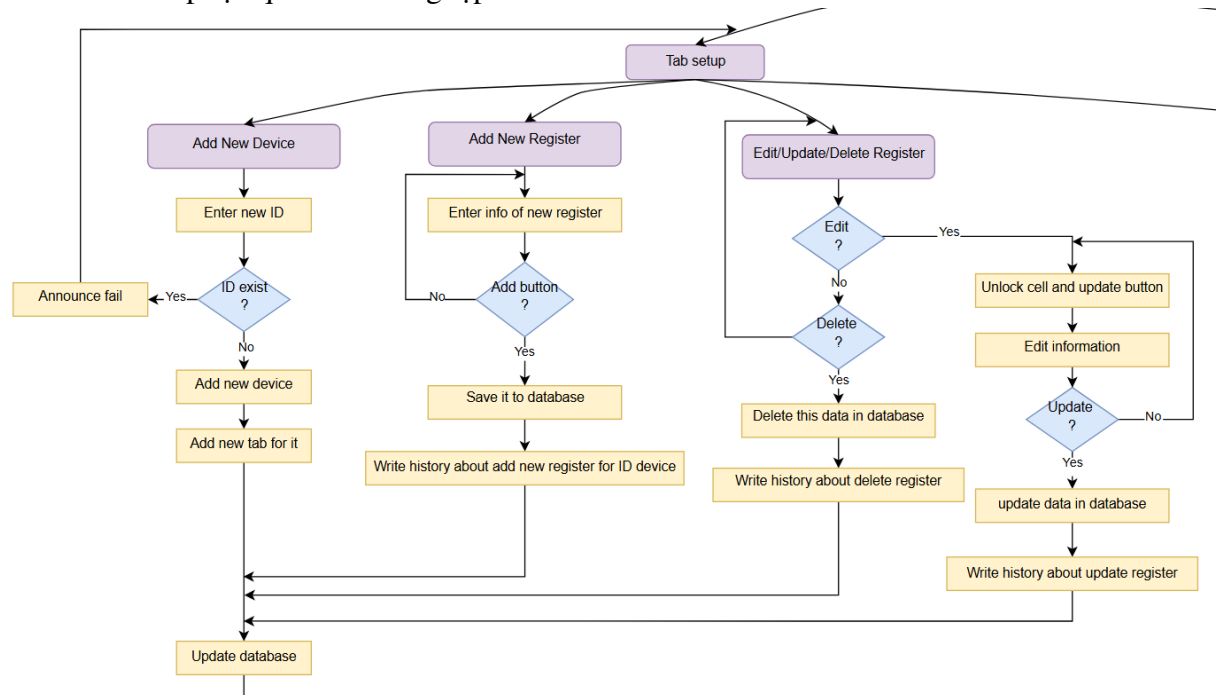


Hình 4-14: Lưu đồ giải thuật phần khởi động App

Quá trình này diễn ra khi người dùng bắt đầu mở ứng dụng:

- **Connect signal:** Gắn liên kết nút nhấn với hàm thực hiện chức năng của nút nhấn.
- **Init tab window on UI:** Khởi tạo các khung hình có trên App.

- **Init controller block-Logic:** Khởi tạo các hàm xử lý dữ liệu chính của App.
- **Init Database if not have:** Khởi tạo file để lưu trữ dữ liệu nếu chưa có trong thư mục.
- **Load data from database on table::** cập nhật các dữ liệu đã định sẵn lên App người dùng có thể thấy được.
- **App Loop User Interface:** bước vào vòng lặp khi người dùng bắt đầu tương tác với App.
- **Stop app:** Kiểm tra có tắt app không.
Yes: Tiến hành đóng App và kết thúc vòng lặp.
No: Tiếp tục quá trình vòng lặp.



Hình 4-15: Lưu đồ giải thuật phần tab Setup

Chức năng tạo mới thiết bị:

- **Enter new ID:** Nhập mới một ID của một thiết bị mới vào hệ thống.
- **ID exist?:** Kiểm tra ID đã tồn tại trong hệ thống chưa.
Yes: In ra thông báo lỗi và quay về giao diện của tab setup
No - Add new device: Tạo mới một thiết bị trong hệ thống.
- **Add new tab for it:** tạo cho thiết bị mới này 1 tab riêng trên cửa sổ.
- **Update database:** Cập nhật lại số lượng và danh sách thanh ghi lưu trong

database.

Chức năng thêm mới thanh ghi:

- **Enter info of new register:** Nhập các số liệu liên quan tới thanh ghi vào các ô.
- **Add button?:** Kiểm tra ID đã bấm nút Add Register trên giao diện chưa.
No: Quay về tiếp tục nhập.
Yes: Tiến hành lưu các thông tin trong ô nhập liệu.
- **Save it to database:** Cập nhập lại số lượng và danh sách thanh ghi lưu trong database.
- **Write history about add new register for ID device:** tiến hành tạo dòng log ở tab history về việc vừa thêm một thanh ghi vào hệ thống.

Các chức năng phụ kèm theo Edit/Update/Delete:

Chức năng nút Edit:

- **Edit?:** Kiểm tra nút nhấn Edit có được nhấn không.
Yes:
- **Unlock cell and update button:** Mở khóa các ô trong bảng và cho phép người dùng thay đổi thông tin.
- **Edit information:** Người dùng chỉnh sửa thông tin. **No:** Kiểm tra tiếp nút nhấn Delete.

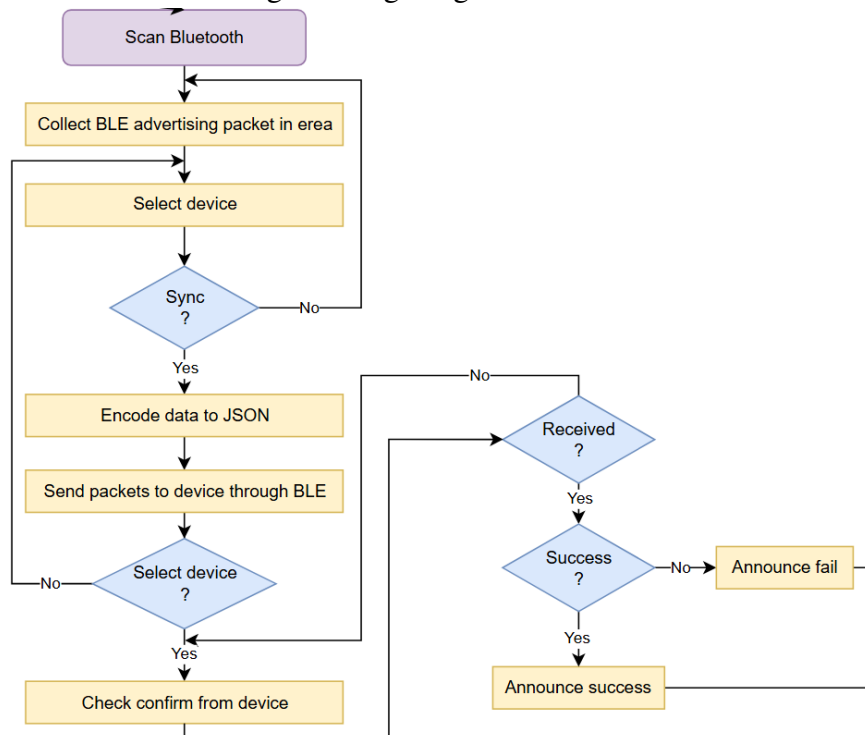
Chức năng nút Delete:

- **Delete?:** Kiểm tra nút nhấn Delete có được nhấn không.
No: tiếp tục chỉnh sửa thông tin trong bảng.
Yes:
- **Delete this data in database:** Xóa dòng người dùng vừa chọn trong database
- **Write history about delete register:** tiến hành tạo dòng log ở tab history về việc vừa xóa một thanh ghi.

Chức năng nút Update:

- **Update?:** Kiểm tra nút nhấn Update có được nhấn không.
No: tiếp tục chỉnh sửa thông tin trong bảng.
Yes:

- **Update data in database:** Cập nhập lại các thông tin vừa chỉnh sửa vào database.
- **Write history about update register:** tiến hành tạo dòng log ở tab history về việc vừa chỉnh sửa thông tin trong bảng.

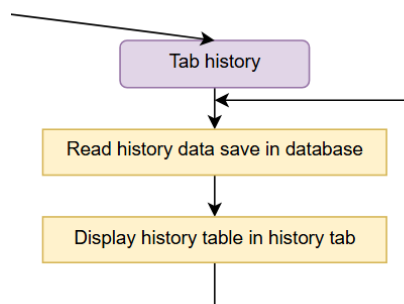


Hình 4-16: Lưu đồ giải thuật phần chức năng Sync

Chức năng này giúp đóng gói dữ liệu gồm các thanh ghi mà người dùng nhập từ máy tính và gửi xuống cho thiết bị thực hiện thu thập dữ liệu từ các thanh ghi và các thông số đã nhập trên App:

- **Collect BLE advertising packet in area:** tìm các thiết bị xung quanh hiện đang phát tín hiệu Bluetooth trong không gian.
- **Select device:** Chọn thiết bị mong muốn gửi dữ liệu.
- **Sync?:** Kiểm tra nút gửi dữ liệu đã được nhấn chưa.
No: Nếu chưa thì tiếp tục tìm các thiết bị và chọn thiết bị.
Yes: nếu đã nhấn nút sync.
- **Encode data to JSON:** Đóng gói toàn bộ dữ liệu thanh ghi theo dạng JSON.
- **Send packets to device through BLE:** chia nhỏ gói tin ra và gửi lần lượt xuống thiết bị

- **Select device?:** Kiểm tra đã chọn thiết bị muốn gửi dữ liệu chưa.
No: Thông báo lỗi và yêu cầu chọn thiết bị trước khi gửi.
(Yes:) Tiến hành gửi dữ liệu và đợi xác nhận từ thiết bị.
- **Received?:** Đợi nhận gói tin phản hồi, nếu chưa có tiếp tục chờ.
- **Success?:** Kiểm tra phản hồi là thành công hay thất bại.
No: In thông báo lỗi ra màn hình.
Yes: Thông báo đã gửi thành công cho thiết bị.



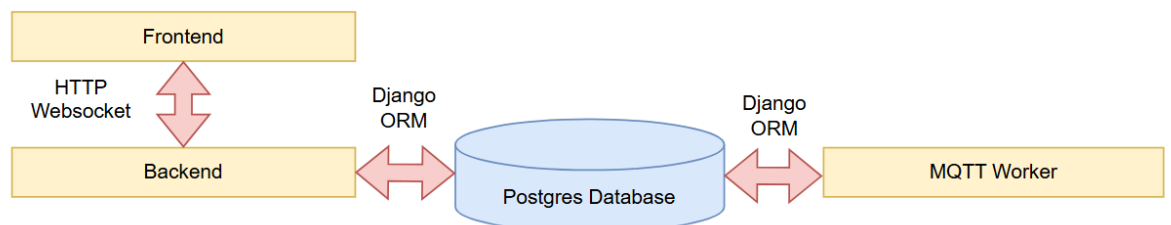
Hình 4-17: Lưu đồ giải thuật phần tab History

Sử dụng chủ yếu để hiển thị lịch sử nhập liệu:

- **Read history data save in database:** Đọc các log đã lưu từ những chức năng khác thêm vào.
- **Display history table in history tab:** Hiển thị các nội dung log vừa đọc dưới dạng bảng ở tab history.

4.3 Phát triển giao diện Dashboard theo dõi dữ liệu

4.3.1 Thiết kế phần mềm

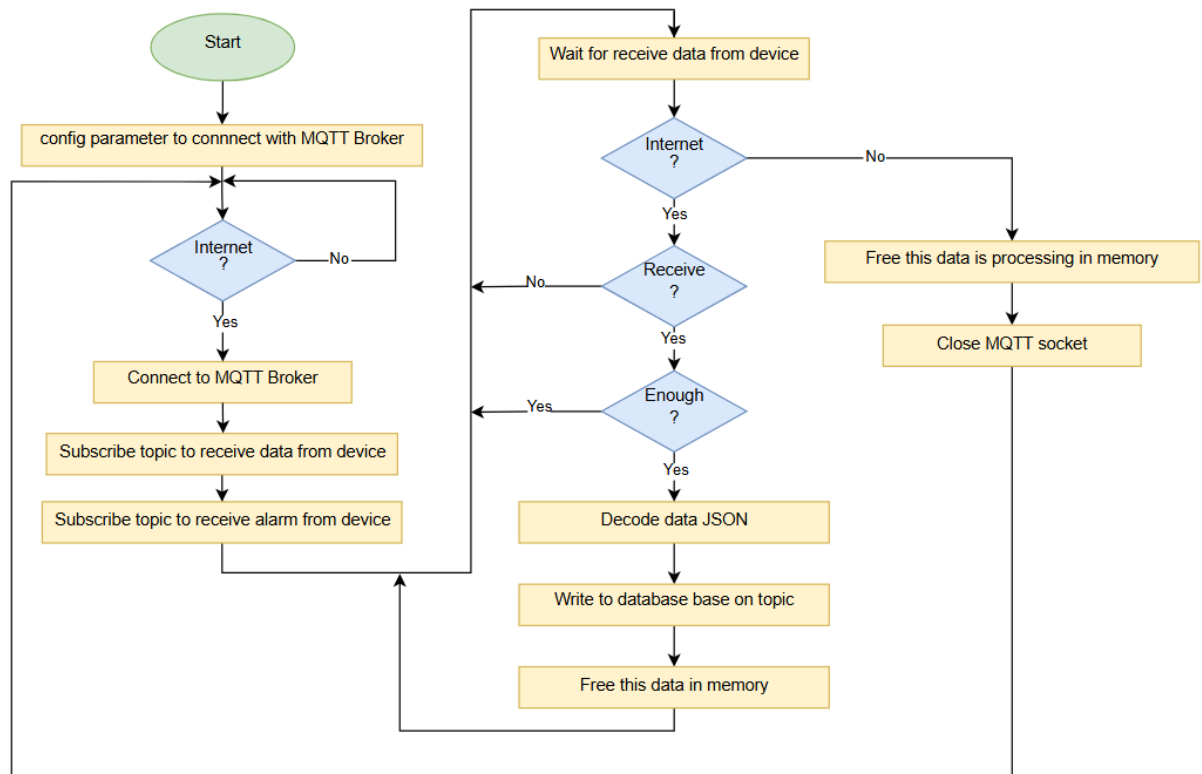


Hình 4-18: Sơ đồ tổng quát các phần của trang web

Trang web được phát triển dựa trên một Repo đã có sẵn, chạy trên localhost, trang web này dùng để cập nhật dữ liệu theo thời gian thực lên giao diện người dùng thông

qua Websocket hoặc HTTP, sử dụng database Postges làm nơi lưu trữ thông tin, giao diện người dùng được xây dựng dựa trên thư viện React, phần backend sử dụng framework Django.

Trong đồ án này ta xây dựng thêm một tiến trình chạy ngầm song song với backend để thu thập dữ liệu được gửi lên từ thiết thông qua MQTT. Tiến trình này sẽ thực hiện các công việc cụ thể như đợi gói tin, phân tích gói tin, lưu dữ liệu sau khi đóng gói vào database của trang web. Với mục đích là hiển thị các thông tin thật lên trang web cho người dùng nhìn trực quan hơn về dữ liệu mà người dùng thu thập được thay vì phải mở terminal hay chỉ xem được tại thiết bị.



Hình 4-19: Lưu đồ giải thuật phần nhận dữ liệu từ thiết bị - MQTT Worker

Chức năng chính là đăng ký vào các Topic mà thiết bị đã tạo và đợi nhận dữ liệu được gửi từ thiết bị lên MQTT Broker.

- **Config parameter to connect with MQTT Broker:** Khởi tạo các thông số liên quan đến giao tiếp MQTT như địa chỉ của broker, cổng kết nối.
- **Internet?:** Kiểm tra tình trạng kết nối mạng của máy chủ.
No: Tiếp tục chờ tới khi có mạng.

Yes: Bắt đầu quá trình nhận dữ liệu từ thiết bị.

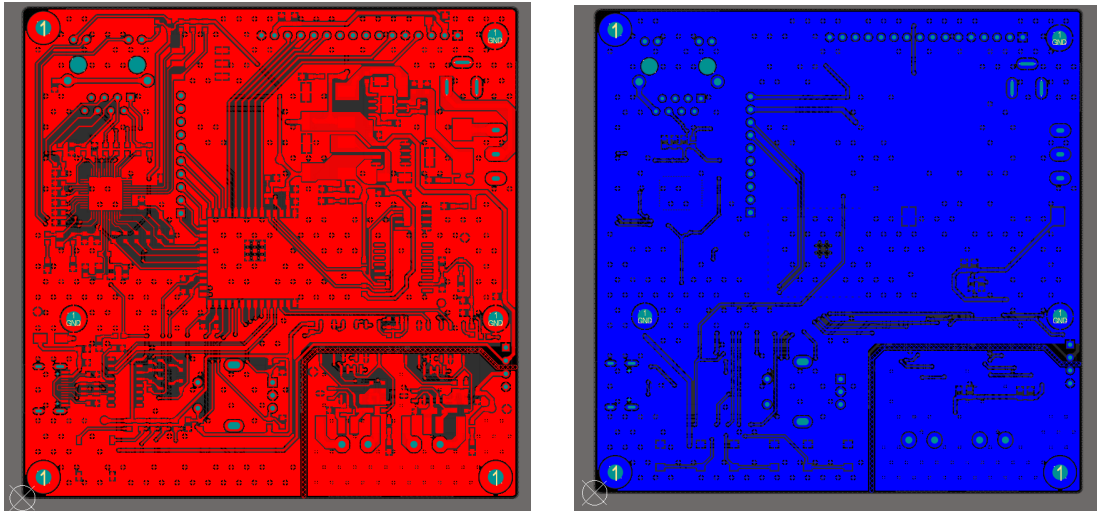
- **Connect to MQTT Broker:** Thực hiện kết nối với MQTT broker đã được định nghĩa trước.
- **Subscribe topic to receive data from device:** Đăng ký vào topic nhận dữ liệu đồng hồ điện.
- **Subscribe topic to receive alarm from device:** Đăng ký vào topic nhận cảnh báo khi phát hiện đường truyền lỗi.
- **Wait for receive data from device:** đợi các gói tin thông qua các Topic đã đăng ký với MQTT broker.
- **Internet?:** Kiểm tra tình trạng kết nối mạng hiện tại.
No: Mất kết nối mạng
- **Free this data is proccessing in memory:** Giải phóng vùng nhớ chứa các nội dung đang xử lý của MQTT.
- **Close MQTT socket:** Tắt socket hiện tại tại cổng 1883.
Yes: Nếu có kết nối mạng.
- **Receice?:** Kiểm tra đã bắt được dữ liệu từ MQTT broker không.
No: Tiếp tục quay lại đợi nhận gói tin.
Yes: Kiểm tra tình trạng gói tin.
- **Enough?:** Kiểm tra tình trạng dữ liệu từ message đã đủ chưa.
No: Tiếp tục nhận các gói tin khác.
Yes: Bắt đầu phân tích gói tin.
- **Decode data JSON:** giải mã gói tin JSON.
- **Write to database base on topic:** Tiến hành lưu dữ liệu vào đúng database của web dựa trên topic và thông tin định danh của thiết bị gửi lên.
- **Free this data in memory:** Sau khi đã lưu dữ liệu vào database thì tiến hành dọn dẹp vùng nhớ cũ và tiếp tục kiểm tra và đợi nhận gói tin tiếp theo.

CHƯƠNG 5 KẾT QUẢ THU ĐƯỢC

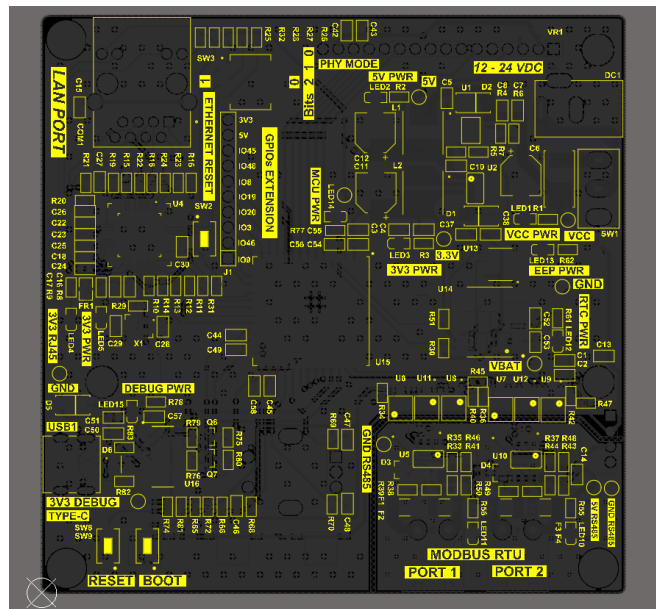
5.1 Kết quả thực hiện phần cứng thực tế

5.1.1 Kết quả thiết kế phần cứng trên phần mềm

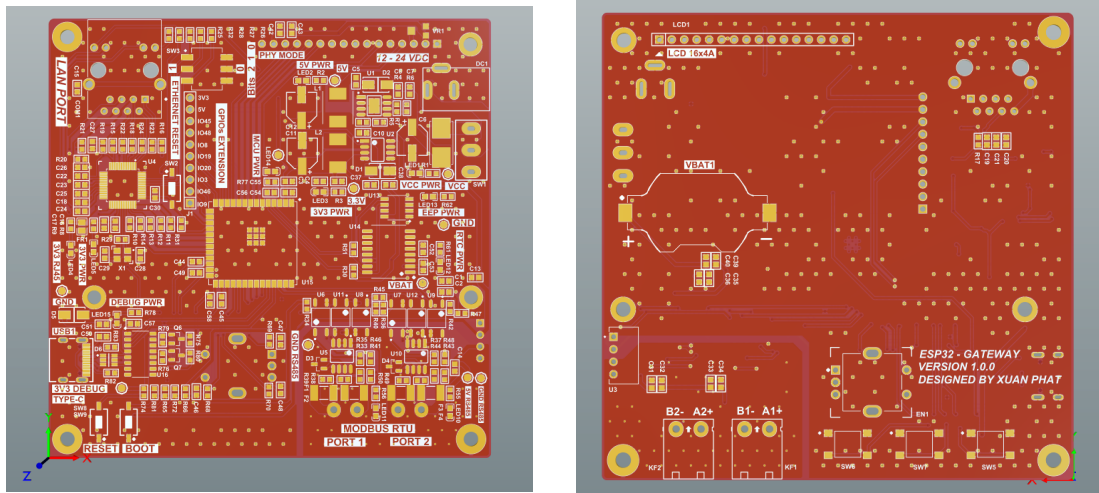
Thực hiện sắp xếp linh kiện hợp lý trên mạch PCB hai lớp, với các linh kiện được bố trí chủ yếu ở lớp Top. Khối tạo nguồn cách ly 5V, hai port sử dụng cho Modbus RTU, Màn hình LCD và nút nhấn đặt ở lớp Bottom.



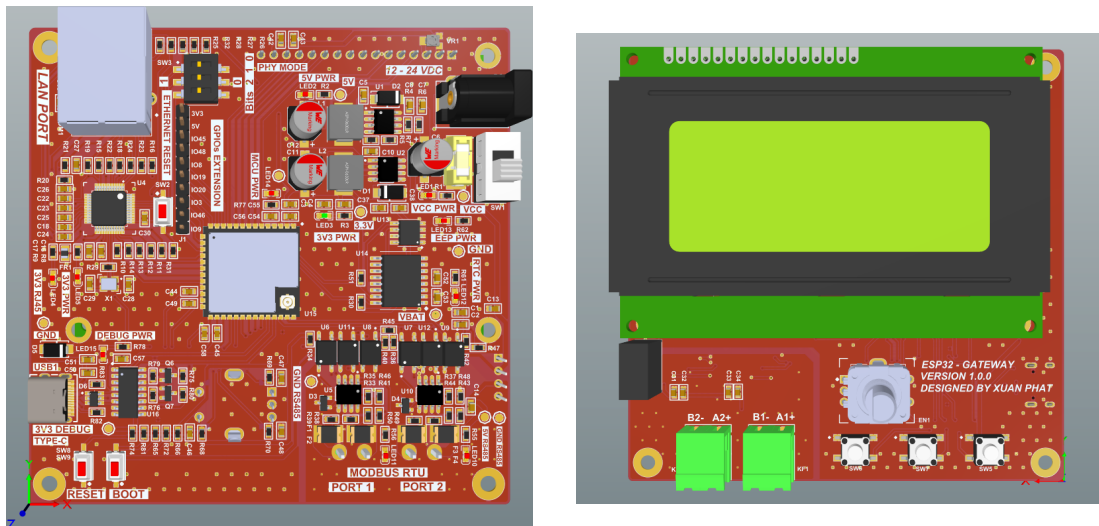
Hình 5-1: Hình ảnh mạch 2D sau khi đã đi dây ở lớp Top và lớp Bottom



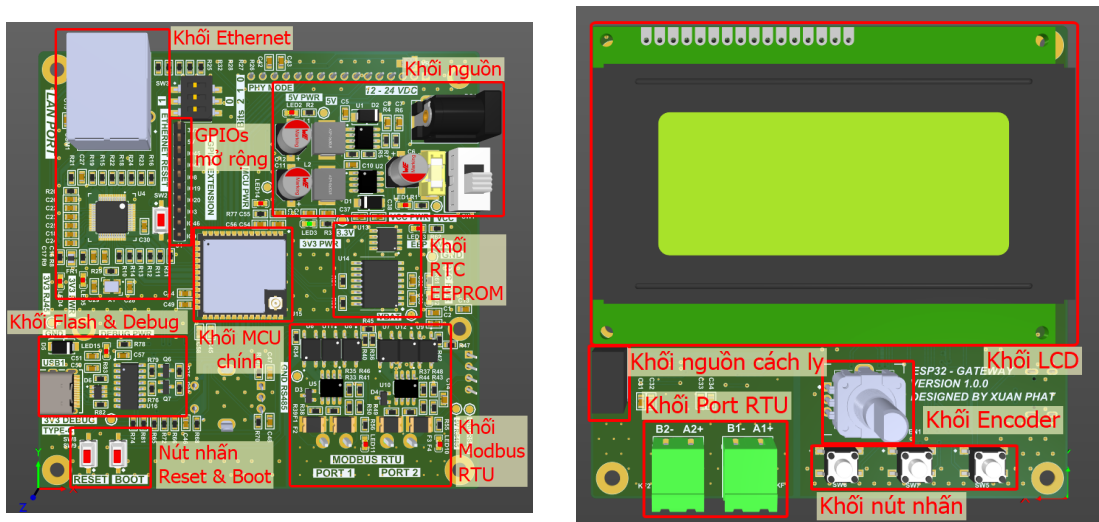
Hình 5-2: phần đặt các ký hiệu trên bề mặt của lớp Top



Hình 5-3: Hình ảnh mạch 3D khi chưa đặt linh kiện của lớp Top và lớp Bottom

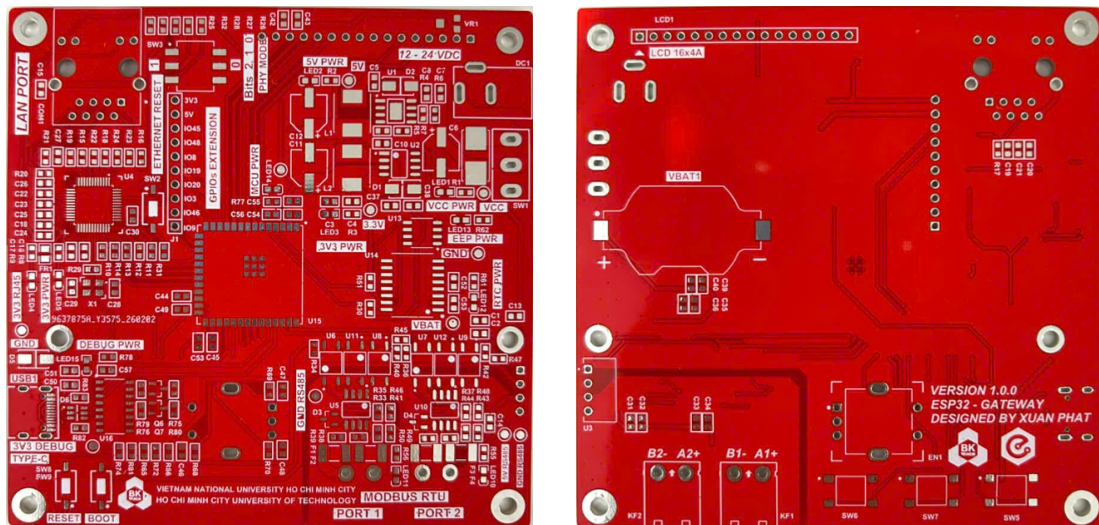


Hình 5-4: Hình ảnh mạch 3D khi đã đặt linh kiện của lớp Top và lớp Bottom

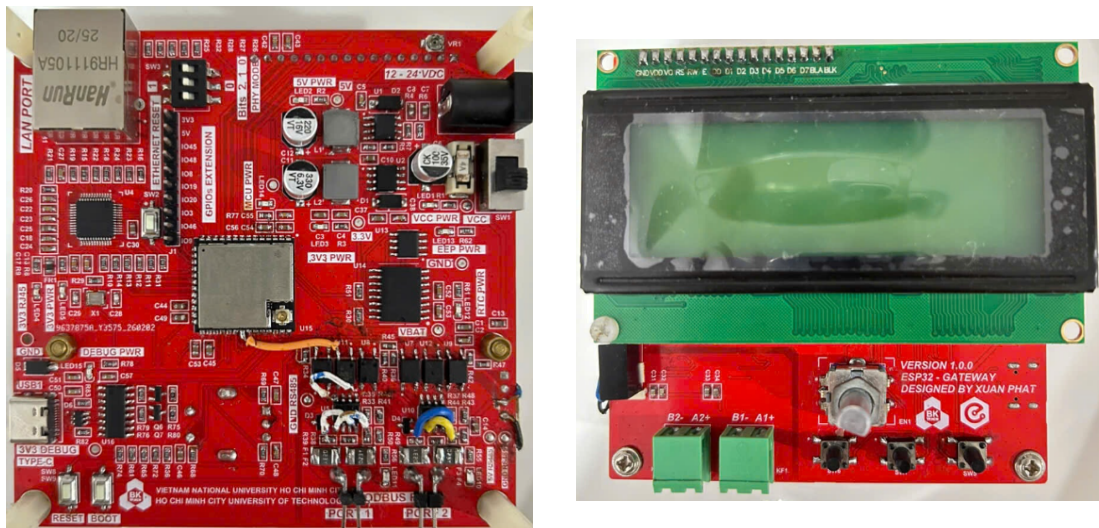


Hình 5-5: Các khối chức năng trên mạch sau khi sắp xếp

5.1.2 Thực hiện thi công mạch thực tế



Hình 5-6: Mạch khi chưa hàn linh kiện



Hình 5-7: Mạch sau khi đã hàn các linh kiện lên mạch

Mạch thực tế sau khi hàn các linh kiện và sửa các lỗi chưa được phát hiện khi vẽ schematic chi tiết.

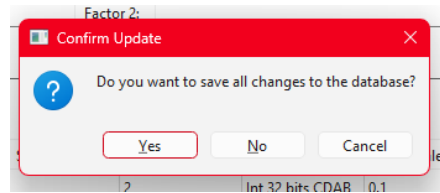
5.2 Kết quả phần mềm App nhập dữ liệu

ID	Parameter	Unit	Function	Start Address	Quantity	Data Type	Scale	Multiplier	Factor 1	Factor 2	Action
1	P_Total_Fwd-...	W	Read Holding ...	0	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
2	P_L1_Fwd (Acti...	W	Read Holding ...	2	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
3	P_L2_Fwd-...	W	Read Holding ...	4	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
4	P_L3_Fwd-...	W	Read Holding ...	6	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
5	P_Total_Rev-...	W	Read Holding ...	8	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
6	P_L1_Rev-...	W	Read Holding ...	10	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
7	P_L2_Rev-...	W	Read Holding ...	12	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
8	P_L3_Rev-...	W	Read Holding ...	14	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
9	Q_Total-(Total-...	VAr	Read Holding ...	16	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
10	Q_L1-(Reactive-...	VAr	Read Holding ...	18	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
11	Q_L2-(Reactive-...	VAr	Read Holding ...	20	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete
12	Q_L3-(Reactive-...	VAr	Read Holding ...	22	2	Int 32 bits CDAB	0.1	Data * Scale	NULL	NULL	Delete

Hình 5-8: Tab nhập liệu chính khi vừa khởi động App

Hình 5-9: Box nhập ID cho thiết bị mới

Hình 5-10: Thông báo lỗi khi ID đã tồn tại và thông báo lỗi khi chưa chọn thiết bị



Hình 5-11: Thông báo có cập nhật dữ liệu mới không

Setup	History

Hình 5-12: Tab lịch sử của App

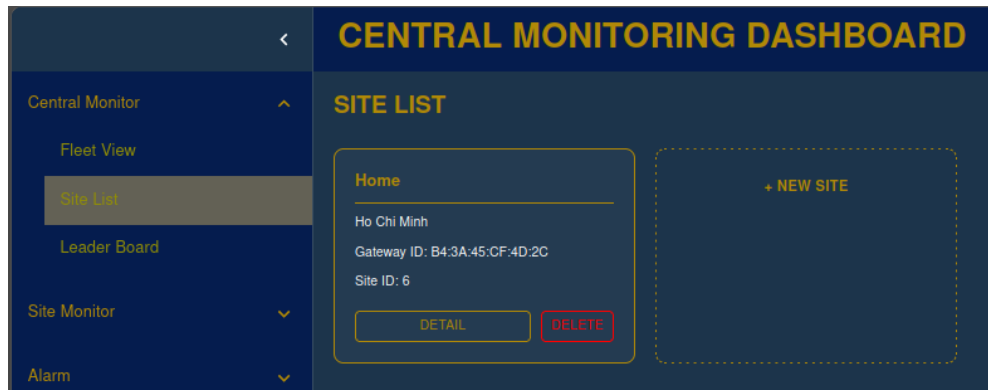
Name	Date modified	Type	Size
08052026.CSV		Microsoft Excel Com...	680 KB
09052026.CSV		Microsoft Excel Com...	830 KB

15413	13/05/2026 17:47:39	1	V_L12_Avg	154	426.1	V
15414	13/05/2026 17:47:39	1	V_L23_Avg	155	426.2	V
15415	13/05/2026 17:47:39	1	V_L31_Avg	156	424.4	V
15416	13/05/2026 17:47:39	1	V_L2_Dmd	157	281.1	V
15417	13/05/2026 17:47:39	1	V_L23_Dmd	158	281.2	V
15418	13/05/2026 17:47:39	1	V_L31_Dmd	159	280	V
15419	13/05/2026 17:47:39	1	I_L1_Max	159	28	A
15420	13/05/2026 17:47:39	1	I_L2_Max	161	3.38	A
15421	13/05/2026 17:47:39	1	I_L3_Max	162	3.35	A
15422	13/05/2026 17:47:39	1	I_L1_Avg	163	1.9	A
15423	13/05/2026 17:47:39	1	I_L2_Avg	164	1.88	A
15424	13/05/2026 17:47:39	1	I_L3_Avg	165	1.9	A
15425	13/05/2026 17:47:39	4	VTR	4000	1	-
15426	13/05/2026 17:47:39	4	CTR	4001	1	-
15427	13/05/2026 17:47:39	4	Voltage-L1-N	4002	245.5	V
15428	13/05/2026 17:47:39	4	Voltage-L2-N	4003	245.6	V
15429	13/05/2026 17:47:39	4	Voltage-L3-N	4004	244.4	V
15430	13/05/2026 17:47:39	4	Voltage-L1-N-Avg	4005	245.9	V
15431	13/05/2026 17:47:39	4	Voltage-L2-N-Avg	4006	245.8	V
15432	13/05/2026 17:47:39	4	Voltage-L3-N-Avg	4007	244.4	V
15433	13/05/2026 17:47:39	4	Voltage-L1-N-Min	4008	0	V
15434	13/05/2026 17:47:39	4	Voltage-L2-N-Min	4009	0	V
15435	13/05/2026 17:47:39	4	Voltage-L3-N-Min	4010	0	V
15436	13/05/2026 17:47:39	4	Voltage-L1-N-Max	4011	249.2	V
15437	13/05/2026 17:47:39	4	Voltage-L2-N-Max	4012	248.3	V
15438	13/05/2026 17:47:39	4	Voltage-L3-N-Max	4013	246.9	V
15439	13/05/2026 17:47:39	4	Voltage-L1-L2	4014	425	V
15440	13/05/2026 17:47:39	4	Voltage-L2-L3	4015	424	V
15441	13/05/2026 17:47:39	4	Voltage-L3-L1	4016	424	V

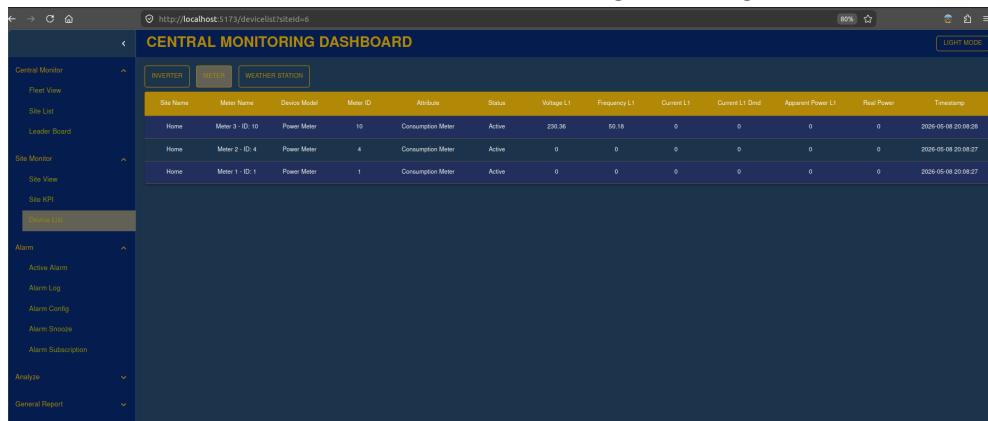
Hình 5-13: Lưu giữ liệu vào thẻ nhớ SD Card và lưu dữ liệu theo từng ngày

Thời gian được đọc từ RTC, thực hiện tạo tên file mới theo định dạng ngày tháng năm, so sánh với các tên file hiện có trong thẻ nhớ, nếu đã tồn tại thì tiếp tục ghi vào các file cũ, nếu chưa có thì tạo một file mới với tên file là ngày tháng năm.

5.3 Kết quả phát triển giao diện Dashboard

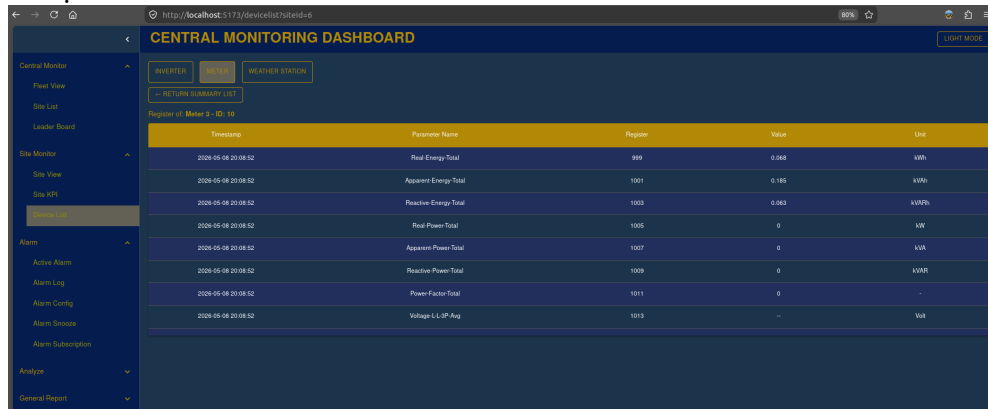


Hình 5-14: Hiển thị các site có trong hệ thống của user



Hình 5-15: Dữ liệu các đồng hồ điện hiển thị tổng quát trên trình duyệt web

Cung cấp hai nút nhấn giúp người dùng có thể xem được thông các đồng hồ điện bên trong site đó và nếu site đó không dùng thì có thể xóa hoàn toàn. Trong đồ án này ta thực hiện gán một thiết bị thu thập dữ liệu cho một site thông qua dải ID có trên thiết bị khởi tạo.



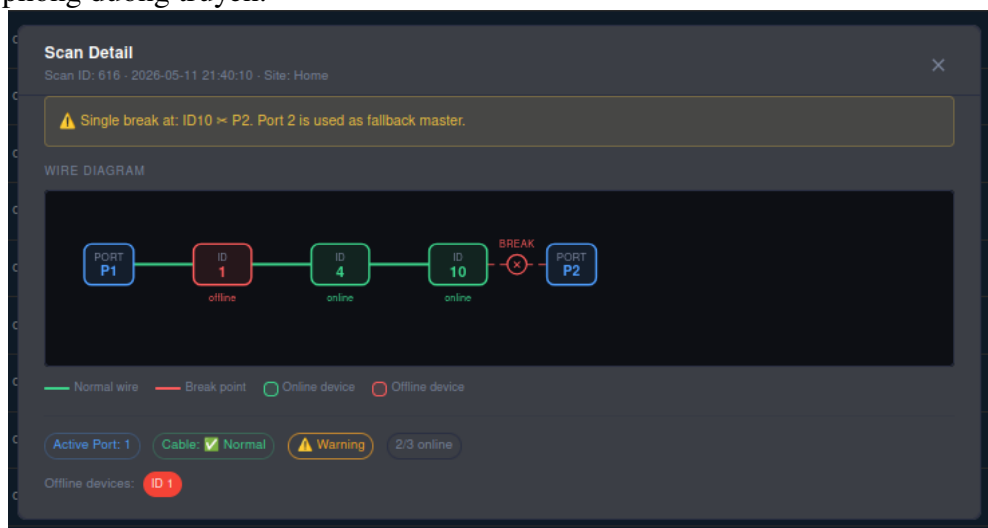
Hình 5-16: Dữ liệu từng thanh ghi của từng đồng hồ điện

Khi người dùng nhấp vào dòng của một đồng hồ bất kì thì dữ liệu chi tiết của đồng hồ điện đó hiện ra từng thanh ghi. Các thanh ghi nhập từ App sẽ xuất hiện toàn bộ trong trang này.

CENTRAL MONITORING DASHBOARD						
Central Monitor						
Site Monitor						
Alarm						
Active Alarm						
Alarm Log						
Alarm Config						
Alarm Settings						
Device	Site	Alarm Content	Reason	Comment	Scan Time	
2/3 online	Home	1 device(s) disconnected	Line Broken Port 1	Warning	11/05/2026, 21:53:24	
2/3 online	Home	1 device(s) disconnected	Line Broken Port 1	Warning	11/05/2026, 21:52:44	
2/3 online	Home	1 device(s) disconnected	Line Broken Port 1	Warning	11/05/2026, 21:52:05	
2/3 online	Home	1 device(s) disconnected	Line Broken Port 1	Warning	11/05/2026, 21:51:25	

Hình 5-17: Bảng tổng quát các cảnh báo đã gửi lên từ thiết bị

Tại đây các cảnh báo liên quan đến các sự cố đường truyền được thiết bị phát hiện trong quá trình thu thập dữ liệu hoặc phát hiện khi người dùng sử dụng nút nhấn chức năng trên thiết bị để thực hiện kiểm tra đường trường. Khi người dùng nhấp vào một cảnh báo bất kì thì trang web sẽ mô phỏng lại đường truyền kèm theo các lỗi cụ thể trên đường truyền tại thời điểm thiết bị gửi tin nhắn cảnh báo. Backend của trang web tiến hành đăng ký nhận tin nhắn cảnh báo từ thiết bị và tiến hành chờ, dữ liệu sau khi nhận từ MQTT Broker sẽ được lưu vào database của trang web và được gửi dữ liệu lên frontend thông qua Websocket, frontend dựa trên dữ liệu nhận được thì sẽ tiến hành mô phỏng đường truyền.



Hình 5-18: Hiện thị cụ thể thông tin đường truyền dựa trên Alarm gửi lên

Thực hiện mô phỏng lại đường truyền và các thiết bị hiện có trên đường truyền. Quá trình khởi tạo hình ảnh mô phỏng được thực hiện bằng thư viện React ngay trên phần quản lý của Frontend.

CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã hoàn thành việc thiết kế và thi công một hệ thống Gateway giám sát năng lượng điện dựa trên vi điều khiển ESP32-S3, đáp ứng đầy đủ các yêu cầu đề ra. Về phần cứng, thiết bị được thiết kế trên bo mạch PCB hai lớp tích hợp đầy đủ các khối chức năng: khối nguồn hạ áp ổn định, khối giao tiếp RS485 cách ly quang học, khối Ethernet sử dụng chip W5500, khối lưu trữ EEPROM và RTC, cùng giao diện người dùng LCD 16x04 với nút nhấn và encoder. Về phần mềm nhúng, hệ thống sử dụng FreeRTOS để quản lý đa nhiệm, cho phép thu thập dữ liệu định kỳ từ các đồng hồ đo điện qua giao thức Modbus RTU, lưu trữ dữ liệu vào thẻ nhớ SD Card khi mất kết nối mạng, và đồng bộ dữ liệu lên Cloud thông qua giao thức MQTT khi có kết nối. Ứng dụng cấu hình trên máy tính được xây dựng bằng PyQt cho phép kỹ thuật viên nhập các thông số thanh ghi và đồng bộ cấu hình xuống thiết bị qua Bluetooth BLE một cách thuận tiện. Trang web Dashboard phát triển trên nền React và Django hiển thị dữ liệu thời gian thực từ các đồng hồ điện, đồng thời cung cấp tính năng cảnh báo và mô phỏng trực quan trạng thái đường truyền Modbus. Nhìn chung, hệ thống vận hành ổn định và đáp ứng được nhu cầu thực tế của các doanh nghiệp trong việc giám sát và quản lý năng lượng điện theo thời gian thực.

6.2 Hướng phát triển

Trong tương lai, hệ thống có thể được cải tiến và mở rộng theo nhiều hướng để nâng cao tính ứng dụng và độ tin cậy. Thứ nhất, về phần cứng, có thể tích hợp thêm tính năng cập nhật firmware từ xa (OTA – Over The Air) giúp việc bảo trì và nâng cấp hệ thống trở nên thuận tiện hơn mà không cần tiếp cận trực tiếp thiết bị, đồng thời có thể mở rộng số lượng cổng RS485 để hỗ trợ thu thập dữ liệu từ nhiều đường dây Modbus song song. Thứ hai, về phần mềm và bảo mật, cần bổ sung cơ chế mã hóa dữ liệu truyền qua MQTT bằng giao thức TLS/SSL và xác thực người dùng trên trang web nhằm đảm bảo an toàn thông tin trong môi trường công nghiệp. Thứ ba, trang web Dashboard có thể được nâng cấp với các tính năng phân tích dữ liệu nâng cao như vẽ biểu đồ xu hướng tiêu thụ điện năng theo thời gian, tính toán chỉ số hiệu quả năng lượng (PUE) và xuất báo cáo tự động theo định kỳ. Cuối cùng, hệ thống có thể tích hợp thêm trí tuệ nhân tạo để phát hiện bất thường trong dữ liệu điện năng, từ đó cảnh báo

sớm các sự cố tiềm ẩn trước khi chúng ảnh hưởng đến quy trình sản xuất của doanh nghiệp.